Documentation

signotec signoPAD-API/Web

Components for communicating with signotec Sigma, Zeta, Omega, Gamma, Delta and Alpha LCD pads from a web application

Version: 3.2.1
Date: 13.01.2021

© signotec GmbH
www.signotec.de
Tel.: +49 (0) 2102 53575 10
E-mail: info@signotec.de

Contents

LEGAL NOTICE **7**

1 DOCUMENT HISTORY **8**

2 FUNCTIONAL OVERVIEW **9**

3 SYSTEM REQUIREMENTS FOR WINDOWS **10**

4 SYSTEM REQUIREMENTS FOR LINUX **11**

5 INSTALLATION OPTIONS **12**

5.1 OPTION 1 (LOCAL.SIGNOTECWEBSOCKET.DE) 12
5.2 OPTION 2 (LOCAL.SIGNOTECWEBSOCKET.DE, LOCALHOST AND 127.0.0.1) 12
5.3 OPTION 3 (LOCALHOST AND 127.0.0.1) 13
5.4 OPTION 4 (127.0.0.1 WITHOUT TLS) 13
5.5 ADD-ON FOR INTERNET EXPLORER 14
5.6 ENABLE EDGE SUPPORT (ALLOW LOOPBACK) 14
5.7 REMOVING THE OLD CA CERTIFICATE FROM THE LIST OF TRUSTED ROOT CERTIFICATION AUTHORITIES 14
5.8 FREE CONFIGURATION OF THE CONNECTION URL 14
5.9 SILENT INSTALLATION 15
5.10 INSTALLATION UNDER LINUX 15

6 LOCAL CONNECTION TYPE OF PADS **17**

6.1 USER-SPECIFIC CONFIGURATION FILE FOR CONNECTION TYPE 17
6.2 SYSTEM-WIDE CONFIGURATION FILE FOR CONNECTION TYPE 18
6.3 INSTALLATION OF 'PADCONFIG.INI' 18

7 STANDARD AND API MODE **19**

7.1 STANDARD MODE 19
7.2 API MODE 19

8 INTEGRATION VIA JAVASCRIPT LIBRARY **20**

8.1 PARAMETERS AND RETURN VALUES OF THE FUNCTIONS 20
8.2 CALLBACKS 20
8.3 PROMISES 20
8.3.1 Use of promise objects 21
8.3.2 Chaining of promise objects 21
8.4 STPADSERVERLIBCOMMONS NAMESPACE 22
8.4.1 createConnection 22
8.4.2 destroyConnection 22
8.4.3 getInterfaceVersion 23
8.4.4 setInterfaceVersion 23
8.4.5 getServerVersion 24
8.4.6 handleDisconnect 24
8.4.7 handleNextSignaturePoint 25
8.4.8 handleLogging 25
8.5 STPADSERVERLIBDEFAULT NAMESPACE 26
8.5.1 searchForPads 26
8.5.2 openPad 27
8.5.3 startSelectionDialog 27
8.5.4 startSignature 28
8.5.5 retrySignature 32
8.5.6 stopSignature 33
8.5.7 confirmSignature 33
8.5.8 cancelSignature 34

8.5.9 getSignatureData 34
8.5.10 getSignatureImage 35
8.5.11 getSigningCert 35
8.5.12 closePad 36
8.5.13 handleError 36
8.5.14 handleCancelSignature 37
8.5.15 handleRetrySignature 37
8.5.16 handleConfirmSignature 37
8.5.17 handleCancelSelection 38
8.5.18 handleConfirmSelection 38
8.5.19 handleSelectionChange 38
8.6 STPADSERVERLIBAPI NAMESPACE 39
8.6.1 Special features of parameters 39
8.6.2 Device.setComPort 40
8.6.3 Device.getConnectionType 41
8.6.4 Device.getComPort 41
8.6.5 Device.getIpAddress 41
8.6.6 Device.getCount 42
8.6.7 Device.getInfo 42
8.6.8 Device.getVersion 42
8.6.9 Device.getCapabilities 42
8.6.10 Device.open 43
8.6.11 Device.close 43
8.6.12 Sensor.getSampleRateMode 43
8.6.13 Sensor.setSampleRateMode 43
8.6.14 Sensor.setSignRect 44
8.6.15 Sensor.clearSignRect 44
8.6.16 Sensor.setScrollArea 44
8.6.17 Sensor.setPenScrollingEnabled 44
8.6.18 Sensor.addHotSpot 45
8.6.19 Sensor.addScrollHotSpot 45
8.6.20 Sensor.setHotSpotMode 45
8.6.21 Sensor.clearHotSpots 46
8.6.22 Sensor.handleHotSpotPressed 46
8.6.23 Sensor.handleDisplayScrollPosChanged 46
8.6.24 Signature.getResolution 46
8.6.25 Signature.start 47
8.6.26 Signature.stop 47
8.6.27 Signature.confirm 47
8.6.28 Signature.retry 47
8.6.29 Signature.cancel 48
8.6.30 Signature.getSignData 48
8.6.31 Signature.saveAsStreamEx 48
8.6.32 Signature.getBounds 48
8.6.33 Signature.getState 49
8.6.34 Display.erase 49
8.6.35 Display.eraseRect 49
8.6.36 Display.configPen 50
8.6.37 Display.setFont 50
8.6.38 Display.setFontColor 50
8.6.39 Display.setTarget 50
8.6.40 Display.setText 51
8.6.41 Display.setTextInRect 51
8.6.42 Display.setImage 51
8.6.43 Display.setImageFromStore 52

8.6.44 Display.setOverlayRect 52
8.6.45 Display.setScrollPos 52
8.6.46 Display.getScrollPos 52
8.6.47 Display.saveImageAsStream 53
8.6.48 Display.setStandbyImage 53
8.6.49 Display.configSlideShowEx 53
8.6.50 Display.getStandbyId 54
8.6.51 Display.getWidth 54
8.6.52 Display.getHeight 54
8.6.53 Display.getResolution 54
8.6.54 Display.getTargetWidth 54
8.6.55 Display.getTargetHeight 55
8.6.56 Display.getScrollSpeed 55
8.6.57 Display.setScrollSpeed 55
8.6.58 Display.getRotation 55
8.6.59 Display.setRotation 56

9 INTEGRATION AT PROTOCOL LEVEL 57

9.1 TYPES (TOKEN_TYPE KEY) 57
9.1.1 Requests (TOKEN_TYPE_REQUEST value) 57
9.1.2 Responses (TOKEN_TYPE_RESPONSE value) 59
9.1.3 Events (TOKEN_TYPE_SEND value) 59
9.2 COMMANDS (TOKEN_CMD AND TOKEN_CMD_ORIGIN KEYS) 60
9.2.1 TOKEN_CMD_GET_INTERFACE_VERSION value 60
9.2.2 TOKEN_CMD_SET_INTERFACE_VERSION value 60
9.2.3 TOKEN_CMD_GET_SERVER_VERSION value 61
9.2.4 TOKEN_CMD_SEARCH_FOR_PADS value 62
9.2.5 TOKEN_CMD_OPEN_PAD value 62
9.2.6 TOKEN_CMD_SELECTION_DIALOG value 63
9.2.7 TOKEN_CMD_SELECTION_CHANGE value 64
9.2.8 TOKEN_CMD_SELECTION_CONFIRM value 64
9.2.9 TOKEN_CMD_SELECTION_CANCEL value 64
9.2.10 TOKEN_CMD_SIGNATURE_START value 65
9.2.11 TOKEN_CMD_SIGNATURE_RETRY value 66
9.2.12 TOKEN_CMD_SIGNATURE_STOP value 67
9.2.13 TOKEN_CMD_SIGNATURE_CONFIRM value 67
9.2.14 TOKEN_CMD_SIGNATURE_CANCEL value 67
9.2.15 TOKEN_CMD_SIGNATURE_SIGN_DATA value 68
9.2.16 TOKEN_CMD_SIGNATURE_IMAGE value 69
9.2.17 TOKEN_CMD_SIGNING_CERT value 69
9.2.18 TOKEN_CMD_CLOSE_PAD value 70
9.2.19 TOKEN_CMD_INCORRECT_COMMAND value 70
9.2.20 TOKEN_CMD_SIGNATURE_POINT value 70
9.2.21 TOKEN_CMD_DISCONNECT value 71
9.2.22 TOKEN_CMD_ERROR value 71
9.2.23 TOKEN_CMD_API_DEVICE_SET_COM_PORT value 71
9.2.24 TOKEN_CMD_API_DEVICE_GET_CONNECTION_TYPE value 72
9.2.25 TOKEN_CMD_API_DEVICE_GET_COM_PORT value 72
9.2.26 TOKEN_CMD_API_DEVICE_GET_IP_ADDRESS value 72
9.2.27 TOKEN_CMD_API_DEVICE_GET_COUNT value 72
9.2.28 TOKEN_CMD_API_DEVICE_GET_INFO value 73
9.2.29 TOKEN_CMD_API_DEVICE_GET_VERSION value 73
9.2.30 TOKEN_CMD_API_DEVICE_GET_CAPABILITIES value 73
9.2.31 TOKEN_CMD_API_DEVICE_OPEN value 74
9.2.32 TOKEN_CMD_API_DEVICE_CLOSE value 74

9.2.33 TOKEN_CMD_API_SENSOR_GET_SAMPLE_RATE_MODE value 74
9.2.34 TOKEN_CMD_API_SENSOR_SET_SAMPLE_RATE_MODE value 74
9.2.35 TOKEN_CMD_API_SENSOR_SET_SIGN_RECT value 75
9.2.36 TOKEN_CMD_API_SENSOR_CLEAR_SIGN_RECT value 75
9.2.37 TOKEN_CMD_API_SENSOR_SET_SCROLL_AREA value 75
9.2.38 TOKEN_CMD_API_SENSOR_SET_PEN_SCROLLING_ENABLED value 76
9.2.39 TOKEN_CMD_API_SENSOR_ADD_HOT_SPOT value 76
9.2.40 TOKEN_CMD_API_SENSOR_ADD_SCROLL_HOT_SPOT value 76
9.2.41 TOKEN_CMD_API_SENSOR_SET_HOT_SPOT_MODE value 77
9.2.42 TOKEN_CMD_API_SENSOR_CLEAR_HOT_SPOTS value 77
9.2.43 TOKEN_CMD_API_SENSOR_HOT_SPOT_PRESSED value 77
9.2.44 TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION value 77
9.2.45 TOKEN_CMD_API_SIGNATURE_START value 78
9.2.46 TOKEN_CMD_API_SIGNATURE_STOP value 78
9.2.47 TOKEN_CMD_API_SIGNATURE_CONFIRM value 78
9.2.48 TOKEN_CMD_API_SIGNATURE_RETRY value 79
9.2.49 TOKEN_CMD_API_SIGNATURE_CANCEL value 79
9.2.50 TOKEN_CMD_API_SIGNATURE_GET_SIGN_DATA value 79
9.2.51 TOKEN_CMD_API_SIGNATURE_SAVE_AS_STREAM_EX value 80
9.2.52 TOKEN_CMD_API_SIGNATURE_GET_BOUNDS value 80
9.2.53 TOKEN_CMD_API_SIGNATURE_GET_STATE value 80
9.2.54 TOKEN_CMD_API_DISPLAY_ERASE value 81
9.2.55 TOKEN_CMD_API_DISPLAY_ERASE_RECT value 81
9.2.56 TOKEN_CMD_API_DISPLAY_CONFIG_PEN value 81
9.2.57 TOKEN_CMD_API_DISPLAY_SET_FONT value 81
9.2.58 TOKEN_CMD_API_DISPLAY_SET_FONT_COLOR value 82
9.2.59 TOKEN_CMD_API_DISPLAY_SET_TARGET value 82
9.2.60 TOKEN_CMD_API_DISPLAY_SET_TEXT value 82
9.2.61 TOKEN_CMD_API_DISPLAY_SET_TEXT_IN_RECT value 83
9.2.62 TOKEN_CMD_API_DISPLAY_SET_IMAGE value 83
9.2.63 TOKEN_CMD_API_DISPLAY_SET_IMAGE_FROM_STORE value 83
9.2.64 TOKEN_CMD_API_DISPLAY_SET_OVERLAY_RECT value 84
9.2.65 TOKEN_CMD_API_DISPLAY_SET_SCROLL_POS value 84
9.2.66 TOKEN_CMD_API_DISPLAY_GET_SCROLL_POS value 84
9.2.67 TOKEN_CMD_API_DISPLAY_SAVE_IMAGE_AS_STREAM value 85
9.2.68 TOKEN_CMD_API_DISPLAY_SET_STANDBY_IMAGE value 85
9.2.69 TOKEN_CMD_API_DISPLAY_CONFIG_SLIDE_SHOW_EX value 85
9.2.70 TOKEN_CMD_API_DISPLAY_GET_STANDBY_ID value 86
9.2.71 TOKEN_CMD_API_DISPLAY_GET_WIDTH value 86
9.2.72 TOKEN_CMD_API_DISPLAY_GET_HEIGHT value 86
9.2.73 TOKEN_CMD_API_DISPLAY_GET_RESOLUTION value 86
9.2.74 TOKEN_CMD_API_DISPLAY_GET_TARGET_WIDTH value 87
9.2.75 TOKEN_CMD_API_DISPLAY_GET_TARGET_HEIGHT value 87
9.2.76 TOKEN_CMD_API_DISPLAY_GET_SCROLL_SPEED value 87
9.2.77 TOKEN_CMD_API_DISPLAY_SET_SCROLL_SPEED value 87
9.2.78 TOKEN_CMD_API_DISPLAY_GET_ROTATION value 88
9.2.79 TOKEN_CMD_API_DISPLAY_SET_ROTATION value 88
9.2.80 TOKEN_CMD_API_DISPLAY_SCROLL_POS_CHANGED value 88
9.3 PARAMETER 88
9.3.1 Special features of parameters in API mode 89
9.3.2 TOKEN_PARAM_RETURN_CODE key 90
9.3.3 TOKEN_PARAM_ERROR_DESCRIPTION key 90
9.3.4 TOKEN_PARAM_ERROR_CONTEXT key 91
9.3.5 TOKEN_PARAM_PAD_SUBSET key 91
9.3.6 TOKEN_PARAM_CONNECTED_PADS key 91

9.3.7 TOKEN_PARAM_PAD_INDEX key 92
9.3.8 TOKEN_PARAM_PAD_TYPE key 92
9.3.9 TOKEN_PARAM_PAD_CONNECTION_TYPE key 92
9.3.10 TOKEN_PARAM_PAD_COM_PORT key 92
9.3.11 TOKEN_PARAM_PAD_IP_ADDRESS key 93
9.3.12 TOKEN_PARAM_PAD_SERIAL_NUMBER key 93
9.3.13 TOKEN_PARAM_PAD_FIRMWARE_VERSION key 93
9.3.14 TOKEN_PARAM_FIELD_NAME key 93
9.3.15 TOKEN_PARAM_CUSTOM_TEXT key 93
9.3.16 TOKEN_PARAM_TEXT_LAYOUT key 93
9.3.17 TOKEN_PARAM_LAYOUT_ID key 94
9.3.18 TOKEN_PARAM_TEXT_BLOCKS key 94
9.3.19 TOKEN_PARAM_TEXT key 94
9.3.20 TOKEN_PARAM_WIDTH key 95
9.3.21 TOKEN_PARAM_HEIGHT key 95
9.3.22 TOKEN_PARAM_FONT_NAME key 95
9.3.23 TOKEN_PARAM_FONT_STYLE key 95
9.3.24 TOKEN_PARAM_MAX_FONT_SIZE key 95
9.3.25 TOKEN_PARAM_FONT_SIZE_ID key 96
9.3.26 TOKEN_PARAM_CONSTANT key 96
9.3.27 TOKEN_PARAM_PAD_ENCRYPTION key 96
9.3.28 TOKEN_PARAM_DOCHASH key 97
9.3.29 TOKEN_PARAM_ENCRYPTION_CERT key 97
9.3.30 TOKEN_PARAM_ENCRYPTION_CERT_ONLY_WHEN_EMPTY key 97
9.3.31 TOKEN_PARAM_PAD_DIALOG_IMAGE key 97
9.3.32 TOKEN_PARAM_SIGNATURE_SIGN_DATA key 98
9.3.33 TOKEN_PARAM_SIGNATURE_RSA_SCHEME key 98
9.3.34 TOKEN_PARAM_SIGNATURE_RSA_SIGNATURE key 98
9.3.35 TOKEN_PARAM_CERT_ID key 98
9.3.36 TOKEN_PARAM_FILE_TYPE key 99
9.3.37 TOKEN_PARAM_PEN_WIDTH key 99
9.3.38 TOKEN_PARAM_FILE key 99
9.3.39 TOKEN_PARAM_SIGNING_CERT key 99
9.3.40 TOKEN_PARAM_EXCEPTION_CAUSE key 100
9.3.41 TOKEN_PARAM_POINT key 100
9.3.42 TOKEN_PARAM_OS key 100
9.3.43 TOKEN_PARAM_VERSION key 101
9.3.44 TOKEN_PARAM_PAD_DISPLAY_WIDTH key 101
9.3.45 TOKEN_PARAM_PAD_DISPLAY_HEIGHT key 101
9.3.46 TOKEN_PARAM_PAD_SAMPLING_RATE key 101
9.3.47 TOKEN_PARAM_PAD_X_RESOLUTION key 102
9.3.48 TOKEN_PARAM_PAD_Y_RESOLUTION key 102
9.3.49 TOKEN_PARAM_PAD_DIALOG_WIDTH key 102
9.3.50 TOKEN_PARAM_PAD_DIALOG_HEIGHT key 102
9.3.51 TOKEN_PARAM_PAD_DISPLAY_RESOLUTION key 103
9.3.52 TOKEN_PARAM_PAD_CAPABILITIES key 103
9.3.53 TOKEN_PARAM_FIELD_LIST key 103
9.3.54 TOKEN_PARAM_FIELD_ID key 103
9.3.55 TOKEN_PARAM_FIELD_TEXT key 103
9.3.56 TOKEN_PARAM_FIELD_CHECKED key 104
9.3.57 TOKEN_PARAM_FIELD_REQUIRED key 104
9.3.58 TOKEN_PARAM_ROTATION key 104
9.3.59 Other keys 104

10 COMMON DEFINITIONS FOR BOTH INTEGRATION VARIANTS 105

10.1 ERROR CODES 105
10.2 LIST FOR PAD SEARCH 105
10.3 MODEL NUMBERS 106
10.4 CONNECTION TYPES 107
10.5 DEVICE PROPERTIES 107
10.6 IMAGE FORMATS 107
10.7 SIGNING BIOMETRIC DATA 108
10.8 PEN WIDTHS 108

Legal notice

All rights reserved. This document and the components it describes are products copyrighted by signotec GmbH, based in Ratingen, Germany. Reproduction of this documentation, in part or in whole, is subject to prior written approval from signotec GmbH. All hardware and software names used are trade names and/or trademarks of their respective manufacturers/owners. Subject to change at any time without notice. We assume no liability for any errors that may appear in this documentation.

1 Document history

Version	Date	Person responsible	Status/note
2.1	05.04.2017	Paul Grütter	Sections 7.2.16–7.2.18, 7.2.24, 7.2.25, 7.2.29, 7.2.40–7.2.46, 7.2.52–7.2.55, 7.3.16 added. Section 7.2.33 deleted. Sections 7.1.1, 7.2.11 changed.
2.2	19.04.2017	Paul Grütter	Section 7.3.6 changed.
2.3	09.05.2017	Valentin Tkachev	Sections 3, 5.2, 5.3 changed.
2.4	30.01.2018	Valentin Tkachev	Sections 3, 4, 5, 5.1, 5.4 changed.
2.5	28.02.2018	Valentin Tkachev	Sections 5.4, 6, 8.2.58, 8.2.59, 8.3.25, 8.3.26, 8.3.27, 8.3.28, 8.3.29, 8.3.30 added. Sections 5.2, 5.5, 8.2.5, 8.3.4, 8.3.14 changed.
2.6	27.03.2018	Valentin Tkachev	Section 5.5 added.
2.7	20.04.2018	Valentin Tkachev	Section 5.6 added. Sections 5.1, 5.7 changed.
2.8	04.05.2018	Paul Grütter	Sections 8.2.5, 8.2.6, 8.2.7, 8.2.8, 8.2.19, 8.3.3, 8.3.27, 8.3.28, 8.3.29, 8.3.30, 8.3.31 added. Sections 8.2.1, 8.3.1 changed.
2.9	11.05.2018	Valentin Tkachev	Sections 2, 5, 5.1, 5.2, 5.3 changed.
2.10	07.06.2018	Paul Grütter	Sections 5.1, 5.2, 5.3 changed.
2.11	11.10.2018	Paul Grütter	Sections 8.2.33, 8.2.34, 8.2.36, 8.2.60–8.2.62, 8.2.69–8.2.72, 8.2.75 added. Sections 8.1.1, 8.1.3, 8.3.1 changed.
2.12	05.03.2019	Philip Höfges / Paul Grütter	Sections 8, 9.2.27, 9.2.70, 9.3.16, 9.3.22, 9.3.29–9.3.31, 10 added. Sections 3, 6, 6.2, 7.2, 9, 9.2.2, 9.2.4, 9.2.9, 9.3.1, 9.3.38–9.3.41 changed.
2.13	04.04.2019	Paul Grütter	Sections 8.4.3, 9.2.1 added. Sections 8.4.1, 8.4.2, 8.4.4, 8.4.5, 9.1.1, 9.3.23 changed.
2.14	05.06.2019	Ulf Bartsch / Paul Grütter	Sections 2, 3, 5, 5.9, 8.4.1, 8.5.4, 8.5.10, 9, 9.2.10, 9.2.15, 9.3.27, 9.3.28, 9.3.53 changed. Sections 5.4, 5.5, 8.4.8, 8.5.11, 9.2.17, 9.3.16–9.3.26, 9.3.32 added. Various examples of code in Section 8 expanded.
3.1.0	12.07.2019	Paul Grütter	New product names 'signoPAD API/Web' introduced. Sections 9.2.19, 9.3.33 added. Sections 8, 9.1.1, 9.1.3 changed.
3.2.0	21.09.2020	Valentin Tkachev	Zeta Pad in Section 9.3.24, 10.2 and 10.3 added. Sections 9.3.30, 9.3.33, 9.3.34, 10.7 added. Sections 8.5.4, 8.5.9, 9.2.10, 9.2.15, 9.3.28, 10.3, 10.5 changed.
3.2.1	13.01.2021	Valentin Tkachev	Sections 10.1, 5.10, 7.2, 8.6 changed.

2 Functional overview

The signotec signoPAD API/Web is signotec's solution for communicating with signotec LCD signature pads from a web application without having to use a browser plug-in. To do so, the product must be installed on the PC on which the web application that would like to communicate with a signature device is launched in a browser.

The signotec signoPAD API/Web consists of the WebSocket Pad Server and the ActiveX Pad Server for Internet Explorer (IE add-on). In general, only one of these components is required. You can find more detailed information on this in the 'Installation options' section.

The WebSocket Pad Server port can be configured as a parameter value at the start. The server can therefore be started as follows, for example:

```
STPadServer.exe 49494
```

Communication takes place using JavaScript Object Notation (JSON); however, there is also a JavaScript library that encapsulates communication.

3 System requirements for Windows

The signotec signoPAD API/Web can be installed on all Windows operating systems from Windows 7 and later. The WebSocket protocol is supported by the following browsers, among others:

- Google Chrome (from Version 4.0.249.0)
- Mozilla Firefox (from Version 24.0)
- Opera (from Version 10.70)
- Microsoft Internet Explorer (from Version 10.0)
- Microsoft Edge

The ActiveX Pad Server is supported exclusively by Internet Explorer. However, it does not require support from the WebSocket protocol.

Information on the proxy server settings:

Please see the 'Installation options' chapter.

Information on SSL encryption:

The signotec WebSocket Pad Server uses its own SSL certificate in order to ensure that it can be called from web applications that use SSL in all browsers. This is set up automatically when the server is installed. The latest versions of Chrome, Firefox, Opera and Edge, as well as the ActiveX Pad Server do not require this in some cases. See also the 'Installation options' section for more information.

Information on the Transport Layer Security (TLS) protocol:

The lowest TLS version supported is 1.1. Please ensure that TLS 1.1 or a new version is enabled in the Windows Internet options ('Advanced' tab). If using Mozilla Firefox, change the settings as follows: Enter `about:config` in the address bar and search for the `security.tls.version.max` or `security.tls.version.min` settings. Both entries must have a value of `1` or higher.

Information on use in terminal server environments:

The WebSocket Pad Server is not supported by Microsoft RDP sessions. If the WebSocket Pad Server is used in a Citrix environment in which there are multiple concurrent users, virtual IP loopback support must be set up for all used browsers and the WebSocket Pad Server.

Further information on setting up the virtual IP loopback can be found at:

https://docs.citrix.com/en-us/xenapp-and-xendesktop/7-15-ltsr/manage-deployment/virtual-ip-virtual-loopback.html

This restriction does not apply to the ActiveX Pad Server.

4 System requirements for Linux

The WebSocket protocol is supported by the following browsers, among others:

- Google Chrome (from Version 4.0.249.0)
- Mozilla Firefox (from Version 24.0)
- Opera (from Version 10.70)

Information on the proxy server settings:

Please see the 'Installation options' chapter.

Information on SSL encryption:

The signotec WebSocket Pad Server uses its own SSL certificate in order to ensure that it can be called from web applications that use SSL. This requires adding the 'signotec GmbH CA' root certificate (root.crt) to the list of trusted root certification authorities.

Information on the Transport Layer Security (TLS) protocol:

The lowest TLS version supported is 1.1. Please ensure that TLS 1.1 or a new version is enabled in the Windows Internet options ('Advanced' tab). If using Mozilla Firefox, change the settings as follows: Enter `about:config` in the address bar and search for the `security.tls.version.max` or `security.tls.version.min` settings. Both entries must have a value of `1` or higher.

5 Installation options

A setup program is available for Windows. The options for the Windows setup are described in the following sections. If you are installing the WebSocket Pad Server under Linux, please refer to the 'Installation under Linux' section.

One of the four options must be selected during installation. The selected option has an impact on the editing of system files, the installation of certificates and the URL under which the signotec WebSocket Pad Server can be addressed and the ActiveX Pad Server installed.

5.1 Option 1 (local.signotecwebsocket.de)

When using this option, the signotec WebSocket Pad Server can only be addressed via the `"local.signotecwebsocket.de"` connection URL. This URL can be changed via a setup property. Please see sections 5.8 and 5.9 for details. The ActiveX Pad Server is not installed by default.

Please note the following limitations:

Internet browser	local.signotecwebsocket.de	localhost	127.0.0.1
Microsoft Edge	supported	not supported	not supported
Microsoft Internet Explorer	supported	not supported	not supported
Mozilla Firefox	supported	supported	supported
Google Chrome	supported	not supported	not supported
Opera	supported	not supported	not supported

The following entry is made in the Windows "hosts" file during installation:

```
127.0.0.1   local.signotecwebsocket.de    # signotec WebSocket Pad Server
```

The 'hosts' file is located in the '%SystemRoot%\System32\drivers\etc' folder. This entry is removed during deinstallation.

When using a proxy server, this must be bypassed for the `local.signotecwebsocket.de` URL. For this reason, when the server is started, an exception is added to the Internet options for this DNS name and removed accordingly upon termination. The entry is located in the `"ProxyOverride"` character string in the registry under the `"HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Internet Settings"` key.

5.2 Option 2 (local.signotecwebsocket.de, localhost and 127.0.0.1)

This option should typically always be used. When using this option, the signotec WebSocket Pad Server can only be addressed via the `"local.signotecwebsocket.de"`, `"localhost"` or `"127.0.0.1"` connection URLs. The first of the URLs stated above can be changed via a setup property. Please see sections 5.8 and 5.9 for details. The ActiveX Pad Server is not installed by default.

Please note the following limitations:

Internet browser	local.signotecwebsocket.de	localhost	127.0.0.1
Microsoft Edge	supported	supported	supported
Microsoft Internet Explorer	supported	supported	supported
Mozilla Firefox	supported	supported	supported
Google Chrome	supported	supported	supported
Opera	supported	supported	supported

All of the settings described in Option 1 must be specified.

When using a proxy server, it must be bypassed for local addresses in order to ensure that `127.0.0.1` can be used. This must be manually configured in the proxy settings of the Windows Internet options or Firefox settings.

5.3 Option 3 (localhost and 127.0.0.1)

This option should only be used if for any reason it must be possible to address the signotec WebSocket Pad Server via `"localhost"` or `"127.0.0.1"`. The ActiveX Pad Server is not installed by default.

Please note the following limitations:

Internet browser	local.signotecwebsocket.de	localhost	127.0.0.1
Microsoft Edge	not supported	supported	supported
Microsoft Internet Explorer	not supported	supported	supported
Mozilla Firefox	not supported	supported	supported
Google Chrome	not supported	supported	supported
Opera	not supported	supported	supported

The Windows 'hosts' file is not changed if this option is used.

When using a proxy server, it must be bypassed for local addresses. This must be manually configured in the proxy settings of the Windows Internet options or Firefox settings.

5.4 Option 4 (127.0.0.1 without TLS)

This option should be used if SSL and root certification are not to be installed. The signotec WebSocket Pad Server can then only be addressed via `"ws://127.0.0.1"`. Communication is not secured. However, the majority of browsers already support this via an SSL-protected web application as it is a local connection. The ActiveX Pad Server is installed by default.

Please note the following limitations:

Internet browser	local.signotecwebsocket.de	localhost	127.0.0.1
Microsoft Edge	not supported	not supported	supported
Microsoft Internet Explorer	not supported	not supported	potentially not supported*
Mozilla Firefox	not supported	not supported	potentially not supported**
Google Chrome	not supported	not supported	supported
Opera	not supported	not supported	supported

* This option is only supported in Internet Explorer if operation of the web application itself is not SSL-protected either. Alternatively, the ActiveX Pad Server can be used for Internet Explorer (see below).

** By default, this option is only supported in Firefox if operation of the web application itself is not SSL-protected either. However, this option can be changed via the configuration switch `network.websocket.allowInsecureFromHTTPS` in `about:config`.

The Windows 'hosts' file is not changed if this option is used.

When using a proxy server, it must be bypassed for local addresses. This must be manually configured in the proxy settings of the Windows Internet options or Firefox settings.

5.5 Add-on for Internet Explorer

A special signotec ActiveX Pad Server add-on is available for Internet Explorer that uses the same protocol as the signotec WebSocket Pad Server. The main advantage of using this component is that it also functions without restriction if it is being used from an SSL-protected web application, whereas the signotec WebSocket Pad Server in this case is only reachable from Internet Explorer via the WSS protocol.

The add-on is only installed by default if installation option 4 is selected, but it can also be selected in the other options in the case of custom installation.

5.6 Enable Edge support (allow loopback)

By default, Edge does not allow localhost connections. The setup file contains the `"ALLOW_EDGE_LOOPBACK"` setup property for loopback activation. The permitted values for the setup property are `"Yes"` to enable localhost connections and no value (`""`) to disable localhost connections. This property can either be silent or enabled/disabled by ticking or unticking the 'Enable Edge support (allow loopback)' box in the GUI. localhost connections are permitted by default.

Further information on Edge localhost connections can be found at:
https://msdn.microsoft.com/en-us/library/windows/apps/Hh780593.aspx

5.7 Removing the old CA certificate from the list of trusted root certification authorities

It is possible that the old CA certificate with the SHA1 `"DD:6C:94:8B:B6:C1:78:DA:AF:28:BD:80:01:4E:41:2A:23:EF:F5:67"` was not removed during the update to the newest version. The WebSocket Pad Server will attempt to remove the certificate when starting. If removing the certificate is undesirable, the setup property `"DONT_REMOVE_OLD_CA_CERT"` should be used with the value `"Yes"` during installation. This property has the value `"No"` by default.

5.8 Free configuration of the connection URL

It is possible to freely configure the connection URL from Version 1.3.2. A setup property, `"WEBSOCKET_URL"`, is available for this purpose. Please see the following section for details.

5.9 Silent installation

A silent installation makes it possible for you to install the signotec signoPAD API/Web without displaying dialog boxes, thus requiring no additional user interaction. To perform a silent installation, start the command prompt. Then, switch to the folder containing the signotec signoPAD API/Web setup file.

CERT_SEL Options	Explanation
"Default"	Installation with option 1 – default value
"DefaultAndLocalhost"	Installation with option 2
"Localhost"	Installation with option 3
"NoTLS"	Installation with option 4
WEBSOCKET_URL	Explanation
"Url"	Installation sets up the WebSocket Pad Server for connection via the URL
ALLOW_EDGE_LOOPBACK Options	Explanation
"Yes"	Edge localhost connections permitted
""	Edge localhost connections not permitted
DONT_REMOVE_OLD_CA_CERT Options	Explanation
"Yes"	Certificate is not removed
""	Certificate is removed
Feature variable	Explanation
WebSocketPadServer	Installation of the WebSocket Pad Server
ActiveX	Installation of the ActiveX Pad Servers (add-on for Internet Explorer)
Documentation	Installation of the documentation
Example	Installation of the example program
PadDrivers	Installation of the signotec HID and WinUSB drivers

The variables are case-sensitive.

Examples:

signotec_signoPAD-API_Web_X.X.X.exe /s /v"/qn CERT_SEL=**\"Default\""**

signotec_signoPAD-API_Web_X.X.X.exe /s /v"/qn CERT_SEL=**\"Localhost\""**

signotec_signoPAD-API_Web_X.X.X.exe /s /v"ADDLOCAL=ALL REMOVE=**Example** /qn"

signotec_signoPAD-API_Web_X.X.X.exe /s /v"/qn ALLOW_EDGE_LOOPBACK=**\"Yes\""**

signotec_signoPAD-API_Web_X.X.X.exe /s /v"/qn ALLOW_EDGE_LOOPBACK=**\"\""**

signotec_signoPAD-API_Web_X.X.X.exe /s /v"/qn ALLOW_EDGE_LOOPBACK=**\"\"** DONT_REMOVE_OLD_CA_CERT=**\"Yes\""**

signotec_signoPAD-API_Web_X.X.X.exe /s /v"/qn WEBSOCKET_URL=**\"test.websocket.url.de\"** ALLOW_EDGE_LOOPBACK=**\"\"** DONT_REMOVE_OLD_CA_CERT=**\"Yes\""**

5.10 Installation under Linux

The signotec signoPAD API/Web consists of the following files:

libSTCPImageEngine.so

libSTPadLib.so

WSSTPadServer

The signotec signoPAD API/Web has the following dependencies:

libpng.so

libtiff.so

libX11.so

libusb-1.0.so

libpthread.so

libboost_system.so

libboost_thread.so

libboost_locale.so

libssl.so

libcrypto.so

Before a connection can be set up between the Web browser and the WebSocket Pad Server, the root.crt certificate must be added to the list of trusted root certification authorities.

The SSL certificate server.pem and its private key serverkey.pem must be located in the same folder with the WSSTPadServer executable file.

The options described in sections 5.1, 5.2 and 5.3 are also valid for the installation of the signotec signoPAD API/Web under Linux.

Depending on the SSL certificates used (sections 5.1, 5.2 and 5.3), a mapping must be added to the 'hosts' file:

127.0.0.1 local.signotecwebsocket.de

When using HID or WinUSB devices, libusb 1.0.16 or higher is also required, which can be downloaded from http://www.libusb.org.

The authorisations for libusb must be adjusted if necessary. For this, the MODE for USB must be changed to 0666:

```
# libusb device nodes
SUBSYSTEM=="usb", ENV{DEVTYPE}=="usb_device", MODE="0666"
```

udev must be restarted afterwards:

```
udevadm control --reload-rules
```

In Debian, this setting is located in the file `/lib/udev/rules.d/50-udev-default.rules` or `/lib64/udev/rules.d/50-udev-default.rules`. The setting may be in a different location when using other distributions.

6 Local connection type of pads

It is possible in Windows to customise the configuration file installed locally on the client to search for pads by connection type. This then overwrites the configuration provided by the web application for the function `searchForPads()` (`"TOKEN_CMD_SEARCH_FOR_PADS"`) command) or `Device.setComPort()` (`"TOKEN_CMD_API_DEVICE_SET_COM_PORT"` command). This feature works with all types of connections. There is a configuration file valid for all machines (practical option when operating terminal servers) and a configuration file valid for all users.

6.1 User-specific configuration file for connection type

There is an optional configuration file that the user can use to set the type of connection to look for when searching for pads. This file should be called '%APPDATA%\signotec\STPadServer\PadConfig.ini'. It will look like this:

```
[CurrentUser]
USB=
SerialPorts=
IPs=
```

The following values are valid for the relevant keys:

Key	Values	Description
`USB`	`Yes`	A search will be made for HID and WinUSB devices.
	`No`	A search will not be made for HID and WinUSB devices.
`SerialPorts`	`None`	A search will not be made on the COM ports or virtual channels for devices.
	`All`	A search will be made for devices on all COM ports and/or virtual channels; this search may take an extremely long time depending on the hardware configuration.
	other	List of numbers separated by a semicolon; a search will be made on each COM port and/or virtual channel with the specified numbers; this search may take an extremely long time if no signotec device is connected to the port.
`IPs`	`None`	A search will not be made on the IP addresses for devices.
	other	List of addresses separated by a semicolon with the structure `"IP4 address:Port"` or `"Host name:Port"`; an Alpha or Delta with an Ethernet connection or an HID device that is connected to a signotec Ethernet USB adapter will be searched for at all specified IP addresses; <Address> is the IP address of the device and has the format x.x.x.x or the host name, <Port> is the port via which the signature device should be activated, i.e. generally 1002. For example, a valid value would therefore be `"192.168.100.100:1002"` or `"host1:1002"`; for details about the signotec Ethernet USB adapter, please refer to your contact at signotec.

If this file exists and it contains at least one validly configured connection type, the signoPAD-API/Web will search for pads on this port or these ports, regardless of what the web

application has defined for the function `searchForPads()` (`"TOKEN_CMD_SEARCH_FOR_PADS"` command) or `Device.setComPort()` (`"TOKEN_CMD_API_DEVICE_SET_COM_PORT"` command).

6.2 System-wide configuration file for connection type

There is an optional configuration file that an administrator can use to select the type of connection for all or only specific users to look for when searching for pads. This file should be called '%PROGRAMDATA%\signotec\STPadServer\PadConfig.ini'. While the file should have the format as seen above, several sections may be used according to the following schema:

Section	Description
`[<Computer name>]`	If the computer name matches one of the available sections, the following keys are valid, regardless of which user is logged on. All other sections are ignored. (This is a useful option for terminal server environments and IP pads where the pad's IP address is not determined based on the user who is signed on but on the client on which the user is currently working. When choosing this option, the 'CLIENTNAME' environment variable on the terminal server must contain the name of the client computer.)
`[<User name>]`	If the user name matches one of the available sections, the following keys are valid. Any existing `[CurrentUser]` section is ignored.
`[CurrentUser]`	The following keys are valid for all users who do not meet one of the criteria listed above.

If this file exists and it contains at least one validly configured connection type, the signotec signoPAD API/Web will search for pads on this port or these ports, regardless of what has been configured in '%APPDATA%\signotec\STPadServer\PadConfig.ini' or what the web application has defined for the function `searchForPads()` (`"TOKEN_CMD_SEARCH_FOR_PADS"` command) or `Device.setComPort()` (`"TOKEN_CMD_API_DEVICE_SET_COM_PORT"` command).

6.3 Installation of 'PadConfig.ini'

The optional system-wide configuration file 'PadConfig.ini' is installed during the setup process under '%PROGRAMDATA%\signotec\STPadServer\' if it is located next to the setup file during installation. This file receives the default privileges of the directory '%PROGRAMDATA%'. It can therefore only be modified by administrators. If the file is not next to the setup file, no files are installed. When the file is uninstalled, any existing file as well as the folder above it (if empty) are deleted. The file is not deleted during an update. It is overwritten while an update is being installed if another file is next to the setup file.

7 Standard and API mode

The signotec signoPAD API/Web can be used in two modes. The standard mode offers an easy-to-use interface with a restricted scope of functions. In API mode, signotec LCD signature pads can be flexibly integrated into a web application of the customer.

7.1 Standard mode

In standard mode, the functions provided by the signotec signoPAD API/Web are sufficient for capturing a signature with a signature device. A given dialog can be displayed on a signature device, and the signature can be captured in a format for real-time display, in SignData format and as an image. Integration into a customer web application using this mode is simple and requires little time and effort.

7.2 API mode

In API mode, the functions provided by the signotec signoPAD API/Web are similar to those of signoPAD API Windows. This mode allows individual integration and use of the extended functionalities of the signotec LCD signature pads.

The functionality of each command is not discussed in detail below; please refer to the signoPAD API Windows documentation, which you can download on the signotec website.

8 Integration via JavaScript library

The signotec signoPAD API/Web comes with a JavaScript library that encapsulates the protocol and asynchrony of the communication, which is why it should preferably be used. As a rule, integration at protocol level should only take place if a JavaScript library cannot be used for technical reasons.

The functions are encapsulated in the library in three namespaces to clearly separate general functions, functions for standard mode and functions for API mode. These namespaces are called `STPadServerLibCommons`, `STPadServerLibDefault` and `STPadServerLibApi`. Standard mode functions can be combined with API mode functions for server versions 3.1.0.0 or later, provided the connection is opened in API mode (`Device.open()`). However, this can lead to unexpected behaviour and should therefore only be done in exceptional cases.

8.1 Parameters and return values of the functions

All functions contained in these libraries follow the same system for parameters and return values.

If there are any parameters, these are passed as a JavaScript object (called `params` in the examples). The mandatory parameters are passed directly when the object is created. Additional parameter values can be set via setters. It is easy to omit parameters that are not mandatory (optional parameters) using this principle.

The return object (called `value` in the examples) consists of key-value pairs and can always contain the following keys:

Key	Type	Description
`command`	String	Associated command for this object; (for differentiation purposes in log outputs, therefore not relevant for the integrator); see also section 'Commands (TOKEN_CMD and TOKEN_CMD_ORIGIN keys)'.
`returnCode`	Number	Error number; `0` means that no error has occurred. A negative value indicates that an error occurred during the process. A positive value has different meanings depending on the command. If the value is negative, the object can also contain the `errorMessage` key; for details, please refer to the section 'Error codes'.
`errorMessage`	String	Error description.

Additional keys can also be included depending on the function, and are documented there.

8.2 Callbacks

Events are implemented via callbacks, which can be implemented by the integrator and called from the library. The names of these callbacks all begin with `handle`. There are also callbacks that can be called in general in the `STPadServerLibCommons` namespace. In the `STPadServerLibDefault` and `STPadServerLibApi` namespaces, these can be only be called when using standard or API mode.

8.3 Promises

A promise is a concept of concurrency. It ensures that processes can be run asynchronously outside the GUI thread. The behaviour is implemented via callbacks, which can be chained.

The library uses the Bluebird implementation (http://bluebirdjs.com). It supports all modern browsers and those that are not compatible with JS6 (for example, Internet Explorer 11). This library is included and must be imported into your own application (see example provided).

8.3.1 Use of promise objects

A promise is a JavaScript object. The object receives a function as a parameter. This function defines the concurrent behaviour. By default, this function has two parameters: `resolve` and `reject`. These are in turn functions that are returned with a value in the event of success (`resolve`) or in the event of an error (`reject`). If the request returns an error from the pad server (negative return value), this is handled in the `reject` function. In the case of JavaScript errors, a corresponding exception is thrown.

Typically, you call the `then()` method on a promise. It receives two functions as parameters, which are internally transferred to the above-mentioned variables `resolve` and `reject`. Here is an example:

```
STPadServerLibDefault.openPad(params)
.then(function(value) {
   // Success. The result can be found inside variable value.
}, function(reason) {
   // Failure. The reason is stored in variable reason.
});
```

8.3.2 Chaining of promise objects

The `openPad (params)` method from the example returns a promise object. Calling `then()` on the promise object controls the subsequent behaviour. Calls to `then()` can be chained. This is how the chaining could look:

```
STPadServerLibDefault.searchForPads(params)
.then(function(value) {
   return STPadServerLibDefault.openPad(params);
}, reject)
.then(function(value) {
    return STPadServerLibDefault.startSignature(params);
}, reject)
.then(null, function(reason) {
    console.error(reason);
});
```

If you return another call to the library in `resolve()`, you can directly use `then()` again on this call, as all library functions return a promise object. This chain can be expanded as desired.

It is strongly recommended that you use the standard function for `reject` (`STPadServerLibCommons.defaultReject`) until the last level of the chain. In this function, the error is passed on from an intermediate step. In the last step, it is then possible to react accordingly. The aim is to avoid executing further steps that will not be successful anyway because of the problem. Should individual handling of `reject` be desired, a promise object must be definitively created and returned so that the chain can continue.

8.4 STPadServerLibCommons namespace

The `STPadServerLibCommons` namespace contains all the functions that can be used in both modes. Parameter objects have the prefix `Params` to be able to distinguish them from the functions.

8.4.1 createConnection

Available from server version 1.0.0.0 onwards.

The `createConnection()` function creates a connection to the WebSocket Pad Server or to the ActiveX Pad Server and needs to be called once before a library function can be used. Depending on the setup configuration, (see the 'Installation options' and 'Add-on for Internet Explorer' sections), the WebSocket Pad server can then be addressed at `"local.signotecwebsocket.de"`, `"localhost"` or `"127.0.0.1"` on port `"49494"` (by default). If Internet Explorer is used, the use of the add-on is always prioritised if it is available.

It does not expect parameter objects, but individual parameters instead and does not return promise objects.

The following parameters are possible:

Key	Type	Description	Default
`url`	String	URL to the WebSocket Pad Server	Mandatory*
`onopen`	String	Name of an `EventListener` that is to be registered as a `WebSocket.onopen` property and is to be called after successful instantiation of the ActiveX Pad Server	Optional
`onclose`	String	Name of an `EventListener` that is to be registered as a `WebSocket.onclose` property	Optional*
`onerror`	String	Name of an `EventListener` that is to be registered as a `WebSocket.onerror` property and is to be called if there is an error in communication with the ActiveX Pad Server	Optional

*Ignored in Internet Explorer if ActiveX Pad Server is available.

Example:

```
STPadServerLibCommons.createConnection("wss://local.signotecwebsocket.de:
49494", onOpen, onClose, onError);
```

8.4.2 destroyConnection

Available from server version 1.0.0.0 onwards.

The `destroyConnection()` function closes a connection to the signoPAD API/Web. This function is also used to close the connection to the signature device, if needed. Functions can then no longer be called without first creating a connection using the function `createConnection()`.

It does not return a promise object.

Example:

```
STPadServerLibCommons.destroyConnection();
```

8.4.3 getInterfaceVersion

Available from server version 1.0.2.4 onwards.

The `getInterfaceVersion()` function returns the set version of the server interface.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

Key	Type	Description
`interfaceVersion`	String	Version number in the format `"major.minor.revision.build"`

Example:

```
STPadServerLibCommons.getInterfaceVersion()
.then(function(value) {
    alert("Server interface version:" + value.interfaceVersion);
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.4.4 setInterfaceVersion

Available from server version 1.0.2.4 onwards.

The `setInterfaceVersion()` function sets the version of the server interface to be used. The version as of which a function/callback or individual parameters are available is documented for each function/callback. It is recommended that you always set the server version for which development is taking place, in order to ensure that the application functions correctly when a newer version of the signoPad API/Web is installed on the client at a later point.

For example, server versions 1.0.0.0 onwards support version 1.0.0.0 of the server interface, while server versions 1.0.2.4 onwards support version 1.0.2.4 of the server interface.

The following keys are possible in the parameter object:

Key	Type	Description	Default
`version`	String	Version number in the format `"major.minor.revision.build"`	Mandatory

No specific keys are possible in the return object.

Example:

```
var params = new
STPadServerLibCommons.Params.setInterfaceVersion("1.6.0.0");
STPadServerLibCommons.setInterfaceVersion(params)
.then(function(value) {
    alert("Interface version has been set.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.4.5 getServerVersion

Available from server version 1.0.2.4 onwards.

The `getServerVersion()` returns the version of the server. This information can be used to make the behaviour of the web application dependent on the server version available.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

Key	Type	Description
`serverVersion`	String	Version number in the format `"major.minor.revision.build"`
`os`	String	Operating system; possible values `"Windows"` or `"Linux"`

Example:

```
STPadServerLibCommons.getServerVersion()
.then(function(value) {
    alert("Server version:" + value.serverVersion + "; OS: " + value.os);
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.4.6 handleDisconnect

Available from server version 1.0.0.0 onwards.

The `handleDisconnect()` callback is called as soon as the connection to a device is interrupted through an external event (for example, unplugging the device). It will only be called for a device that has been opened beforehand using the `STPadServerLibDefault.openPad()` or `STPadServerLibApi.Device.open()` functions.

The following parameters are expected:

Parameter	Type	Description
`padIndex`	Number	Index of the pad to be closed

Example:

```
STPadServerLibCommons.handleDisconnect = function(padIndex) {
    alert("Pad " + padIndex + " was disconnected.");
};
```

8.4.7 handleNextSignaturePoint

Available from server version 1.0.0.0 onwards.

The `handleNextSignaturePoint()` callback is called for each captured point during the signing process. The web application can use this event to render the signature in real time. The resolution of the data is not suitable for use as biometric data. Please use SignData instead, which you can retrieve using the `STPadServerLibDefault.getSignatureData()` or `STPadServerLib.Api.Signature.getSignData()` functions.

The following parameters are expected:

Parameter	Type	Description
`x`	Number	x coordinate between `0` and the display width or horizontal signature resolution of the pad model used (see also `STPadServerLibDefault.openPad()` and `STPadServerLibApi.Signature.getResolution()` functions)
`y`	Number	y coordinate between `0` and the display height or vertical signature resolution of the pad model used (see also `STPadServerLibDefault.openPad()` and `STPadServerLibApi.Signature.getResolution()` functions)
`p`	Number	`0` defines the point of setting down the pen (first point of a signature line).
		`1` defines a pen that is set down (all further points of a signature line).

Example:

```
STPadServerLibCommons.handleNextSignaturePoint = function(x, y, p) {
    alert("New signature point: x = " + x + "; y = " + y + "; p = " + p);
};
```

8.4.8 handleLogging

Available from server version 1.0.0.0 onwards.

The `handleLogging()` callback is called to transfer information that can be logged by the application, if required.

The following parameters are expected:

Parameter	Type	Description
`message`	String	Log message

Example:

```
STPadServerLibCommons.handleLogging = function(message) {
    console.log(message);
};
```

8.5 STPadServerLibDefault namespace

The `STPadServerLibCommons` namespace contains all the functions that can only be used in standard mode. Parameter objects have the prefix `Params` to be able to distinguish them from the functions.

8.5.1 searchForPads

Available from server version 1.0.0.0 onwards.

The `searchForPads()` function starts a search for connected signature devices. If devices are found, their properties are determined and returned in the response.

In Linux, only HID pads are found.

The following keys are possible in the parameter object:

Key	Type	Description	Default
`type`	String	See section 'List for pad search'	`"HID"`

The following specific keys are possible in the return object:

Key	Type	Description
`foundPads`	Array	Array consisting of one object for each device found

The objects in `foundPads` have the following keys:

Key	Type	Description
`index`	Number	Unique ID of the device (as an integer)
`type`	Number	Model number of the device; see section 'Model numbers'
`comPort`	Number	Number of the COM port to which the device is connected
`connectionType`	Number	Connection type of the device; see section 'Connection types'
`firmwareVersion`	String	Firmware version (major.minor) of the device
`ipAddress`	String	IP address of the device
`serialNumber`	String	Serial number of the device
`capabilities`	Number	Bit field describing the properties of a device; see section 'Device properties' (from Version 1.3.0.0 onwards)

Example:

```
var params = new STPadServerLibDefault.Params.searchForPads();
params.setPadSubset("HID");
STPadServerLibDefault.searchForPads(params)
.then(function(value) {
    alert(value.foundPads.length + " pads found.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.5.2 openPad

Available from server version 1.0.0.0 onwards.

The `openPad()` function opens the connection to a device.

The following keys are possible in the parameter object:

Key	Type	Description	Default
`padIndex`	Number	Index of the pad to be opened	Mandatory

The following specific keys are possible in the return object:

Key	Type	Description
`displayWidth`	Number	Width of the display in px
`displayHeight`	Number	Height of the display in px
`displayResolution`	Number	Resolution of the display in dpi (from Version 1.6.0.0 onwards)
`xResolution`	Number	Resolution of the sensor (in px) in x direction (from Version 1.0.2.4 onwards)
`yResolution`	Number	Resolution of the sensor (in px) in y direction (from Version 1.0.2.4 onwards)
`samplingRate`	Number	Sampling rate of the pad in Hz
`dialogWidth`	Number	Width of the signing dialog without toolbar in px (from Version 1.6.0.0 onwards)
`dialogHeight`	Number	Height of the signing dialog without toolbar in px (from Version 1.6.0.0 onwards)

Example:

```
var params = new STPadServerLibDefault.Params.openPad(0);
STPadServerLibDefault.openPad(params)
.then(function(value) {
    alert("Pad opened; samplingRate: " + value.samplingRate);
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.5.3 startSelectionDialog

Available from server version 1.4.0.0 onwards.

The `startSelectionDialog()` function displays a dialog with freely configurable selection fields, as well as 'Cancel' and 'Confirm' buttons, provided that a connection has been opened beforehand using the `openPad()` command. The selection fields are dynamically positioned, depending on the text length and size of the signature device screen, and divided into several pages if necessary. If the value of a selection field is changed by a user through clicking it, a `handleSelectionChanged()` type event is triggered. If the 'Cancel' button is pressed, a `cancelSelection()` type event is triggered. If the 'Confirm' button on the last page is pressed, a `confirmSelection()` type event is triggered. If the 'Confirm' button on a previous page is pressed, the next page is displayed without informing the application. If an error occurs here, the `handleError()` callback is triggered.

The request with the `startSelectionDialog()` command is not implemented in Linux and returns error -25.

The following keys are possible in the parameter object:

Key	Type	Description	Default
`checkboxInformation`	Array	Objects with information about the selection fields described below	Mandatory

The objects in `checkboxInformation` must have the following keys:

Key	Type	Description
`id`	String	ID of a selection field; it can be any value, but may not have already been used for another field.
`text`	String	Text of a selection field; it may have any length and contain `"\n"` for a new paragraph.
`checked`	Boolean	Selection field is initially selected.
`required`	Boolean	Selection field must be selected in order to confirm the selection dialog; if the 'Confirm' button is pressed without all mandatory fields having been selected, the `handleError()` callback is called with error code -400.

No specific keys are possible in the return object.

Example:

```
var params = new STPadServerLibDefault.Params.startSelectionDialog();
params.addCheckboxInformation([{id: "Signature 1", text: "Please sign",
    checked: false, required: true}]);
STPadServerLibDefault.startSelectionDialog(params)
.then(function(value) {
    alert("Please select and confirm elements on the pad.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.5.4 startSignature

Available from server version 1.0.0.0 onwards.

The `startSignature()` function starts the signature capture process, provided a connection has been opened beforehand using the `openPad()` command.

The following keys are possible in the parameter object:

Key	Type	Description	Default
`fieldName`	String	Text that is output at the top left of the signature device's display; usually this should be the name of the signature field that is to be signed.	`null`
`customText`	String	Text that is output at the bottom left of the signature device's display	`null`
`textLayout`	Array	Any texts in a variable layout that are displayed when signing on a signature device; the possible members are described below. (Only available on Windows and only as of version 1.7.0.0)	`null`

`padEncryption`	Boolean	Switches the signoPAD API/Web to 'RSA in pad' mode; before the server is switched to 'RSA in pad' mode, the device properties are checked to determine whether the device supports encryption in the pad. A check is then performed (depending on whether the `encryptionCert` parameter was passed) to determine if the pad has stored a biometric key, or an attempt is made to upload the transferred key to the pad. (Only available on Windows and only as of version 1.3.0.0)	`false`
`docHash`	String	A 20-byte (SHA1) or 32-byte (SHA256) byte array as a Base64 string that is passed to the pad; the hash is encrypted together with the biometric data and may be signed – see section 'Signing biometric data'. Usually this is the checksum of a document to be signed. (Only available on Windows and only as of version 1.3.0.0)	`null`
`encryptionCert`	String	Base64 encoded X.509 certificate with a public key that is loaded into the pad to encrypt the biometric data in the pad. (Only available on Windows and only as of version 1.3.0.0)	`null`
`encryptionCertOnlyWhenEmpty`	Boolean	Specifies whether the public key sent in the parameter `encryptionCert` is only loaded in the pad if the pad does not yet contain one; ignored if the value `encryptionCert` is not set. (Only available on Windows and only as of version 3.2.0.0)	`false`
`dialogImage`	String	Base64 encoded image that is displayed below the toolbar on the signature device's display; the image should be of the size delivered by the `openPad()` function; if either one of the parameters `fieldName` and `customText` is also filled, the text is output on the transferred image and framed by a white rectangle. (Only available on Windows and only as of version 1.6.0.0).	`null`

The objects in `textLayout` can have the following keys:

Key	Type	Description
`id`	Number	Integer ID of the layout. An individual image memory is reserved for every ID in Omega, Gamma, Delta and Alpha devices in which texts can be permanently stored. This allows various layouts with constant texts to be stored in the device. The maximum number of layouts depends on the device and its configuration. If there are no more memories available, the texts will be automatically retransferred with every display.

`textBlocks`	Array	Array of objects that define the text blocks within a layout.

The objects in `textBlocks` can have the following keys:

Key	Type	Description	
`text`	String	Text of a text block. It may have any length and contain "\n" for a new paragraph.	
`width`	Number	Width of a text block in per cent; decimal places are permitted. `100` means that the total available width will be used.	
`height`	Number	Height of a text block in per cent; decimal places are permitted. `100` means that the total available height will be used.	
`fontName`	String	Font of a text block. This font must be installed on the client; otherwise, 'Arial' is used.	
`fontStyle`	Array	Array of objects that define the font style of a text block. `STPadServerLibDefault.FontStyle.BOLD`, `STPadServerLibDefault.FontStyle.ITALIC` and `STPadServerLibDefault.FontStyle.UNDERLINE` are valid values. If the key is missing, contains an invalid value or the style for the configured font does not exist, the standard style of the font is used.	
`maxFontSize`	Number	Maximum size for text output in per cent; decimal places are permitted. `100` is equivalent to 20 pt for Sigma and Zeta devices, and to 40 pt for Gamma, Delta, Omega, and Alpha devices; a smaller size (up to a minimum of 12 pt) is used for long texts in order for the text to be fully displayed.	
`fontSizeId`	Number	Integer ID (optional). All text blocks for which the same ID is used will then use the same font size (the smallest necessary to fit the texts into all text blocks).	
`constant`	Boolean	`true`	The text block rarely changes and is to be permanently stored in the signature device in order to accelerate display. Note: Text blocks marked as constant are not displayed if the parameter `dialogImage` is also used.
		`false`	The text block is to be retransferred in any case (standard).

Note: The values `padEncryption`, `docHash`, `encryptionCert` and `encryptionCertOnlyWhenEmpty` are set with the setter call-up `enablePadEncryption(docHash, encryptionCert, encryptionCertOnlyWhenEmpty)`. The value `padEncryption` is set to `true` in the call-up of `enablePadEncryption`.

No specific keys are possible in the return object.

Examples:

```
var params = new STPadServerLibDefault.Params.startSignature();
params.setFieldName("Customer Signature");
params.setCustomText("Please sign here!");
STPadServerLibDefault.startSignature(params)
.then(function(value) {
    alert("Please sign on the pad.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

```
var params = new STPadServerLibDefault.Params.startSignature();
params.setDialogImage("iVBORw0KGgoAAAANSUhEUgAAAAEAAAABCAIAAACQd1PeAAAAAX
NSR0IArs4c6QAAAARnQU1BAACxjwv8YQUAAAAJcEhZcwAADsMAAA7DAcdvqGQAAAAMSURBVBh
XY/j//z8ABf4C/qc1gYQAAAAASUVORK5CYII=");
params.enablePadEncryption("AAECAwQFBgcICQoLDA0ODxAREhMUFRYXGBkaGxwdHh8="
, "MIICqTCCAZGgAwIBAgIBATANBgkqhkiXy1bpSA==", true);
STPadServerLibDefault.startSignature(params)
.then(function(value) {
    alert("Please sign on the pad.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

```
var params = new STPadServerLibDefault.Params.startSignature();
var textLayout = {};
textLayout.id = 0;
textLayout.textBlocks = [];
textLayout.textBlocks[0] = {};
textLayout.textBlocks[0].text = "I confirm this agreement.";
textLayout.textBlocks[0].width = 100;
textLayout.textBlocks[0].height = 60;
textLayout.textBlocks[0].maxFontSize = 100;
textLayout.textBlocks[0].constant = true;
textLayout.textBlocks[1] = {};
textLayout.textBlocks[1].width = 100;
textLayout.textBlocks[1].height = 7;
textLayout.textBlocks[2] = {};
textLayout.textBlocks[2].text = "Account:";
textLayout.textBlocks[2].width = 32;
textLayout.textBlocks[2].height = 17;
textLayout.textBlocks[2].fontStyle = [];
textLayout.textBlocks[2].fontStyle[0] =
STPadServerLibDefault.FontStyle.UNDERLINE;
textLayout.textBlocks[2].maxFontSize = 100;
textLayout.textBlocks[2].fontSizeId = 0;
textLayout.textBlocks[3] = {};
textLayout.textBlocks[3].width = 2;
textLayout.textBlocks[3].height = 17;
textLayout.textBlocks[4] = {};
textLayout.textBlocks[4].text = "Agreement ID:";
textLayout.textBlocks[4].width = 32;
textLayout.textBlocks[4].height = 17;
textLayout.textBlocks[4].fontStyle = [];
textLayout.textBlocks[4].fontStyle[0] =
STPadServerLibDefault.FontStyle.UNDERLINE;
textLayout.textBlocks[4].maxFontSize = 100;
textLayout.textBlocks[4].fontSizeId = 0;
```

```
textLayout.textBlocks[5] = {};
textLayout.textBlocks[5].width = 2;
textLayout.textBlocks[5].height = 17;
textLayout.textBlocks[6] = {};
textLayout.textBlocks[6].text = "Name:";
textLayout.textBlocks[6].width = 32;
textLayout.textBlocks[6].height = 17;
textLayout.textBlocks[6].fontStyle = [];
textLayout.textBlocks[6].fontStyle[0] =
STPadServerLibDefault.FontStyle.UNDERLINE;
textLayout.textBlocks[6].maxFontSize = 100;
textLayout.textBlocks[6].fontSizeId = 0;
textLayout.textBlocks[7] = {};
textLayout.textBlocks[7].text = "123456";
textLayout.textBlocks[7].width = 32;
textLayout.textBlocks[7].height = 16;
textLayout.textBlocks[7].maxFontSize = 100;
textLayout.textBlocks[7].fontSizeId = 1;
textLayout.textBlocks[8] = {};
textLayout.textBlocks[8].width = 2;
textLayout.textBlocks[8].height = 16;
textLayout.textBlocks[9] = {};
textLayout.textBlocks[9].text = "7890";
textLayout.textBlocks[9].width = 32;
textLayout.textBlocks[9].height = 16;
textLayout.textBlocks[9].maxFontSize = 100;
textLayout.textBlocks[9].fontSizeId = 1;
textLayout.textBlocks[10] = {};
textLayout.textBlocks[10].width = 2;
textLayout.textBlocks[10].height = 16;
textLayout.textBlocks[11] = {};
textLayout.textBlocks[11].text = "John Doe";
textLayout.textBlocks[11].width = 32;
textLayout.textBlocks[11].height = 16;
textLayout.textBlocks[11].maxFontSize = 100;
textLayout.textBlocks[11].fontSizeId = 1;
params.setTextLayout(textLayout);
STPadServerLibDefault.startSignature(params)
.then(function(value) {
    alert("Please sign on the pad.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.5.5 retrySignature

Available from server version 1.0.0.0 onwards.

The `retrySignature()` function discards the signature data without terminating the signature capture process and deletes the rendered signature from the signature device's display. This command will start a new capture process if the signature capture process was terminated beforehand using the `stopSignature()` command.

The function does not expect a parameter object.

No specific keys are possible in the return object.

Example:

```
STPadServerLibDefault.retrySignature()
.then(function(value) {
    alert("Signature has been erased, please sign again.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.5.6 stopSignature

Available from server version 1.0.0.0 onwards.

The `stopSignature()` function terminates the signature capture process that is currently running, and caches the captured signature data.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

Key	Type	Description
`countedPoints`	Number	Number of collected signature points. This can be used to make sure that the signature is sufficiently clear.

Example:

```
STPadServerLibDefault.stopSignature()
.then(function(value) {
    alert(value.countedPoints + " have been captured.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.5.7 confirmSignature

Available from server version 1.0.0.0 onwards.

The `confirmSignature()` function terminates the signature capture process that is currently running, caches the captured signature data and, in contrast to `stopSignature()`, erases the entire device LCD.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

Key	Type	Description
`countedPoints`	Number	Number of collected signature points. This can be used to make sure that the signature is sufficiently clear.

Example:

```
STPadServerLibDefault.confirmSignature()
.then(function(value) {
    alert(value.countedPoints + " have been captured.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.5.8 cancelSignature

Available from server version 1.0.0.0 onwards.

The `cancelSignature()` function terminates the signature capture process that is currently running, discards the signature data and erases the entire device LCD.

The function does not expect a parameter object.

No specific keys are possible in the return object.

Example:

```
STPadServerLibDefault.cancelSignature()
.then(function(value) {
    alert("Signature has been cancelled.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.5.9 getSignatureData

Available from server version 1.0.0.0 onwards.

The `getSignatureData()` function retrieves the SignData of a captured signature, and possibly its digital signature, from the device. SignData is an encrypted, compressed, biometric data format that can be stored in a database or as a tag in a TIFF or PDF document.

The following key is possible in the parameter object:

Key	Type	Description	Default
`rsaScheme`	String	Requires the signature of the biometric data and sets the signature scheme to be used (only evaluated if 'RSA in pad' mode has been activated); see section 'Signing biometric data' (Windows only and only version 3.2.0.0 or later)	`"NONE"`

The following specific keys are possible in the return object:

Key	Type	Description
`signData`	String	Base64 encoded data of the signature in SignData format
`certId`	String	ID of the public key stored in the signature device and used for encryption of the biometric data (if 'RSA in pad' mode has been activated); the ID is a character string consisting of the 'Issuer' and 'Serial Number' (in hexadecimal representation) of the certificate that has been transferred when saving the key, e.g., 'Example certificate (ff 0a)'
`rsaSignature`	String	Base64-coded signature of the biometric data (depending on the value of the `rsaScheme` parameter)

Example:

```
var getSignatureDataParams = new
STPadServerLibDefault.Params.getSignatureData();
getSignatureDataParams.setRsaScheme("PSS");
STPadServerLibDefault.getSignatureData(getSignatureDataParams)
.then(function(value) {
    alert("SignData: " + value.signData + ", CertId: " + value.certId,
          RSASignature: " + value.rsaSignature + ");
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.5.10 getSignatureImage

Available from server version 1.0.0.0 onwards.

The `getSignatureImage()` function delivers a captured signature as image data. The image is created with the aspect ratio of the rectangle that surrounds the signature.

The request with the `getSignatureImage()` command is not implemented in Linux and returns error -25.

The following keys are possible in the parameter object:

Key	Type	Description	Default
`fileType`	Number	Image type; see section 'Image formats'	1
`penWidth`	Number	Line width of the pen (from Version 1.1.0.0 onwards); see section 'Pen widths'	0

The following specific keys are possible in the return object:

Key	Type	Description
`file`	String	The image in Base64-coded form

Example:

```
var params = new STPadServerLibDefault.Params.getSignatureImage();
params.setFileType(STPadServerLibDefault.FileType.PNG);
params.setPenWidth(5);
STPadServerLibDefault.getSignatureImage(params)
.then(function(value) {
    alert("Image: " + value.file);
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.5.11 getSigningCert

Available from server version 1.7.0.0 onwards.

The `getSigningCert()` function retrieves the signature certificate stored in the signature device.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

Key	Type	Description
signingCert	String	The public X.509 certificate saved in the signature device, the corresponding private key of which is used for signing data, in Base64-coded form

Example:

```
STPadServerLibDefault.getSigningCert()
.then(function(value) {
    alert("Signing Cert: " + value.signingCert);
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.5.12 closePad

Available from server version 1.0.0.0 onwards.

The `closePad()` function closes the connection to a device. Before closing, a currently running signature capture process is terminated. Captured signature data is discarded.

The following keys are possible in the parameter object:

Key	Type	Description	Default
padIndex	Number	Index of the pad to be closed	Mandatory

No specific keys are possible in the return object.

Example:

```
var params = new STPadServerLibDefault.Params.openPad(0);
STPadServerLibDefault.closePad(params)
.then(function(value) {
    alert("The pad has been closed.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.5.13 handleError

Available from server version 1.4.0.0 onwards.

The `handleError()` callback is called if an error occurs outside of a request.

The following parameters are expected:

Parameter	Type	Description
errorContext	String	Command in whose context the error has occurred; see also section 'Commands (TOKEN_CMD and TOKEN_CMD_ORIGIN keys)'
errorCode	Number	Error code; for details, please refer to section 'Error codes'
errorDescription	String	Error description.

Example:

```
STPadServerLibDefault.handleError = function(errorContext, errorCode,
errorDescription) {
    alert("Error occured in context " + errorContext + " with return code
     " + returnCode + " and error description " + errorDescription);
};
```

8.5.14 handleCancelSignature

Available from server version 1.0.0.0 onwards.

The `handleCancelSignature()` callback is called if the user presses the 'Cancel' button on the signature device with the pen. The typical reaction should be a call to `cancelSignature()`.

No parameters are expected.

Example:

```
STPadServerLibDefault.handleCancelSignature = function() {
    STPadServerLibDefault.cancelSignature();
};
```

8.5.15 handleRetrySignature

Available from server version 1.0.0.0 onwards.

The `handleRetrySignature()` callback is called if the user presses the 'Retry' button on the signature device with the pen. In the implementation, the typical reaction should be a call to `retrySignature()`.

No parameters are expected.

Example:

```
STPadServerLibDefault.handleRetrySignature = function() {
    STPadServerLibDefault.retrySignature();
};
```

8.5.16 handleConfirmSignature

Available from server version 1.0.0.0 onwards.

The `handleConfirmSignature()` callback is called if the user presses the 'Confirm' button on the signature device with the pen. In the implementation, the typical reaction should be a call to `confirmSignature()`.

No parameters are expected.

Example:

```
STPadServerLibDefault.handleConfirmSignature = function() {
    STPadServerLibDefault.confirmSignature();
};
```

8.5.17 handleCancelSelection

Available from server version 1.4.0.0 onwards.

The `handleCancelSelection()` callback is called if the user presses the 'Cancel' button of a selection dialog on the signature device with the pen. In the implementation the typical reaction should be a call to `closePad()`.

No parameters are expected.

Example:

```
STPadServerLibDefault.handleCancelSelection = function() {
    STPadServerLibDefault.closePad();
};
```

8.5.18 handleConfirmSelection

Available from server version 1.4.0.0 onwards.

The `handleConfirmSelection()` callback is called if the user presses the 'Confirm' button of the last page of a selection dialog on the signature device with the pen. In the implementation the typical reaction should be a call to `startSignature()` or `closePad()`.

No parameters are expected.

Example:

```
STPadServerLibDefault.handleCancelSelection = function() {
    STPadServerLibDefault.closePad();
};
```

8.5.19 handleSelectionChange

Available from server version 1.4.0.0 onwards.

The `handleSelectionChange()` callback is called if the user presses a field of the selection dialog on the signature device with the pen.

The following parameters are expected:

Parameter	Type	Description
`fieldId`	Number	ID of the pressed field
`checked`	Boolean	`TRUE` = field is selected
		`FALSE` = field is not selected

Example:

```
STPadServerLibDefault.handleSelectionChange = function(fieldId, checked)
{
    alert("Selection change: " + fieldId + " -> " + checked);
};
```

8.6 STPadServerLibApi namespace

The `STPadServerLibApi` namespace contains all the functions that can only be used in API mode. The functions and their parameters in API mode are basically very similar to those in signoPAD API. Therefore, this section only lists the names of the functions and the names of the keys in the parameter or return objects; for the meaning of the keys, please refer to the signoPAD API Windows documentation, which you can download on the signotec website.

The functions are divided into a total of four blocks. They are grouped by the function's category. The subsections are named as follows below: `<Group>.<Function>`.

The functions for creating a parameter object have the same name as the function itself and follow this system: `<Group>.Params.<Function>`. They always expect all mandatory parameters; optional parameters can be set via a separate setter. If this setter is not used, an optional parameter has the value defined in the signoPAD API Windows documentation.

Example:

```
var params = new STPadServerLibApi.Device.Params.open(0);
params.setEraseDisplay(false);
STPadServerLibApi.Device.open(params)
.then(function(value) {
    alert("The pad has been opened.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.6.1 Special features of parameters

8.6.1.1 Colours

Parameters that define colours must be passed as a number representing an RGB value. The last-but-two byte determines the share of the blue channel, the last-but-one byte the share of the green channel and the last byte determines the share of the red channel. Therefore, the colour blue corresponds to `"0xFF0000"` or `"16711680"`, the colour green to `"0x00FF00"` or `"65280"` and the colour red to `"0x0000FF"` or `"255"`.

Colour parameter – example:

```
var params = new STPadServerLibApi.Display.Params.configPen(2, 0xFF0000);
STPadServerLibApi.Display.configPen(params)
.then(function(value) {
    alert("The pen has been configured.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.6.1.2 Pointer parameters

Pointers cannot be used. Instead, the data can be passed directly with the request. The data changed by the method is delivered in the return object in the response.

Pointer as an in parameter – example

```
var params = new STPadServerLibApi.Device.Params
    .setComPort("HID;1;4;IP=192.168.100.100:1002");
STPadServerLibApi.Device.setComPort(params)
.then(function(value) {
    alert(value.returnCode + " ports set.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

Pointer as an out parameter – example

```
var params = new STPadServerLibApi.Device.Params.getInfo(0);
STPadServerLibApi.Device.getInfo(params)
.then(function (value) {
    alert("Model: " + value.type + "; Serial: " + value.serial);
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.6.1.3 Byte arrays and images

Instead of byte arrays and images, Base64 strings must be passed and returned. The PNG format is recommended for images.

Passing a PNG image – example

```
var params = new STPadServerLibApi.Display.Params.setImage(0, 0,
"iVBORw0KGgoAAAANSUhEUgAAAAEAAAABCAIAAACQd1PeAAAAAXNSR0IArs4c6QAA
AARnQU1BAACxjwv8YQUAAAAJcEhZcwAADsMAAA7DAcdvqGQAAAAMSURBVBhXY/j/
/z8ABf4C/qc1gYQAAAAASUVORK5CYII=");
STPadServerLibApi.Display.setImage(params)
.then(function (value) {
    alert("Image has been set.");
}, function(reason) {
    alert(reason.errorMessage);
});
```

Return of a byte array – example

```
STPadServerLibApi.Signature.getSignData()
.then(function (value) {
    alert("SignData: " + value.signData);
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.6.1.4 Files

Files (for example, images from the local hard disk) are not supported. Data can only be exchanged as Base64-encoded strings.

8.6.2 Device.setComPort

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `portList`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DeviceSetComPort() method in the signoPAD API.

8.6.3 Device.getConnectionType

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `index`

The following specific keys are possible in the return object:

- `connectionType`

For details and an example, please see the documentation of the DeviceGetConnectionType() method in the signoPAD API.

8.6.4 Device.getComPort

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `index`

The following specific keys are possible in the return object:

- `comPort`

For details and an example, please see the documentation of the DeviceGetComPort() method in the signoPAD API.

8.6.5 Device.getIpAddress

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `index`

The following specific keys are possible in the return object:

- `ipAddress`

For details and an example, please see the documentation of the DeviceGetIPAddress() method in the signoPAD API.

8.6.6 Device.getCount

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `countedDevices`

For details and an example, please see the documentation of the DeviceGetCount() method in the signoPAD API.

8.6.7 Device.getInfo

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `index`

The following specific keys are possible in the return object:

- `serial`
- `type`

For details and an example, please see the documentation of the DeviceGetInfo() method in the signoPAD API.

8.6.8 Device.getVersion

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `index`

The following specific keys are possible in the return object:

- `version`

For details and an example, please see the documentation of the DeviceGetVersion() method in the signoPAD API.

8.6.9 Device.getCapabilities

Available from server version 1.6.0.0 onwards.

The following keys are mandatory in the parameter object:

- `index`

The following specific keys are possible in the return object:

- `capabilities`

For details and an example, please see the documentation of the DeviceGetCapabilities() method in signoPAD API.

8.6.10 Device.open

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `index`

The following setters are available in the parameter object:

- `setEraseDisplay`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DeviceOpen() method in the signoPAD API.

8.6.11 Device.close

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `index`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DeviceClose() method in the signoPAD API.

8.6.12 Sensor.getSampleRateMode

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `sampleRateMode`

The enumeration `SampleRateMode` can be used for the key `sampleRateMode`.

For details and an example, please see the documentation of the SensorGetSampleRateMode() method in the signoPAD API.

8.6.13 Sensor.setSampleRateMode

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `mode`

The enumeration `SampleRateMode` can be used for the key `mode`.

No specific keys are possible in the return object.

For details and an example, please see the documentation of the SensorSetSampleRateMode() method in the signoPAD API.

8.6.14 Sensor.setSignRect

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `left`
- `top`
- `width`
- `height`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the SensorSetSignRect() method in the signoPAD API.

8.6.15 Sensor.clearSignRect

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

No specific keys are possible in the return object.

For details and an example, please see the documentation of the SensorClearSignRect() method in the signoPAD API.

8.6.16 Sensor.setScrollArea

Available from server version 1.5.0.0 onwards.

The following keys are mandatory in the parameter object:

- `left`
- `top`
- `width`
- `height`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the SensorSetScrollArea() method in the signoPAD API.

8.6.17 Sensor.setPenScrollingEnabled

Available from server version 1.5.0.0 onwards.

The following keys are mandatory in the parameter object:

- `enabled`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the SensorSetPenScrollingEnabled() method in the signoPAD API.

8.6.18 Sensor.addHotSpot

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `left`
- `top`
- `width`
- `height`

The following specific keys are possible in the return object:

- `hotspotId`

For details and an example, please see the documentation of the SensorAddHotSpot() method in the signoPAD API.

8.6.19 Sensor.addScrollHotSpot

Available from server version 1.5.0.0 onwards.

The following keys are mandatory in the parameter object:

- `left`
- `top`
- `width`
- `height`
- `type`

The following specific keys are possible in the return object:

- `hotspotId`

For details and an example, please see the documentation of the SensorAddScrollHotSpot() method in the signoPAD API.

8.6.20 Sensor.setHotSpotMode

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `mode`
- `hotspotId`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the SensorSetHotSpotMode() method in the signoPAD API.

8.6.21 Sensor.clearHotSpots

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

No specific keys are possible in the return object.

For details and an example, please see the documentation of the SensorClearHotSpots() method in the signoPAD API.

8.6.22 Sensor.handleHotSpotPressed

Available from server version 1.1.0.0 onwards.

The following parameters are expected:

- `hotspotId`

For details and an example, please see the documentation of the SensorHotSpotPressed() event in the signoPAD API.

8.6.23 Sensor.handleDisplayScrollPosChanged

Available from server version 1.5.0.0 onwards.

The following parameters are expected:

- `xPos`
- `yPos`

For details and an example, please see the documentation of the DisplayScrollPosChanged() event in the signoPAD API.

8.6.24 Signature.getResolution

Available from server version 1.1.0.0 onwards.

The `getResolution()` function delivers the resolution of the signature data that is delivered in the `handleNextSignaturePoint()` callback.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `xResolution`
- `yResolution`

Example:

```
STPadServerLibApi.Signature.getResolution()
.then(function(value) {
    alert("X: " + value.xResolution + "; Y: " + value.yResolution);
}, function(reason) {
    alert(reason.errorMessage);
});
```

8.6.25 Signature.start

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

No specific keys are possible in the return object.

For details and an example, please see the documentation of the SignatureStart() method in the signoPAD API.

8.6.26 Signature.stop

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `countedPoints`

For details and an example, please see the documentation of the SignatureStop() method in the signoPAD API.

8.6.27 Signature.confirm

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `countedPoints`

For details and an example, please see the documentation of the SignatureConfirm() method in the signoPAD API.

8.6.28 Signature.retry

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

No specific keys are possible in the return object.

For details and an example, please see the documentation of the SignatureRetry() method in the signoPAD API.

8.6.29 Signature.cancel

Available from server version 1.1.0.0 onwards.

Although a parameter object is mandatory, it does not have to contain any keys.

The following setters are available in the parameter object:

- `setErase`

The enumeration `EraseOption` can be used for the setter `setErase`.

No specific keys are possible in the return object.

For details and an example, please see the documentation of the SignatureCancel() method in the signoPAD API.

8.6.30 Signature.getSignData

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `signData`

For details and an example, please see the documentation of the SignatureGetSignData() method in the signoPAD API.

8.6.31 Signature.saveAsStreamEx

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `resolution`
- `width`
- `height`
- `fileType`
- `penWidth`
- `penColor`
- `options`

The enumeration `FileType` can be used for the key `fileType`.

The following specific keys are possible in the return object:

- `image`

For details and an example, please see the documentation of the SignatureSaveAsStreamEx() method in the signoPAD API.

8.6.32 Signature.getBounds

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `options`

The following specific keys are possible in the return object:

- `left`
- `top`
- `right`
- `bottom`

For details and an example, please see the documentation of the SignatureGetBounds() method in the signoPAD API.

8.6.33 Signature.getState

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `state`

For details and an example, please see the documentation of the SignatureGetState() method in signoPAD API.

8.6.34 Display.erase

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DisplayErase() method in the signoPAD API.

8.6.35 Display.eraseRect

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `left`
- `top`
- `width`
- `height`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DisplayEraseRect() method in the signoPAD API.

8.6.36 Display.configPen

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `width`
- `color`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DisplayConfigPen() method in the signoPAD API.

8.6.37 Display.setFont

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `name`
- `size`
- `options`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DisplaySetFont() method in the signoPAD API.

8.6.38 Display.setFontColor

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `color`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DisplaySetFontColor() method in the signoPAD API.

8.6.39 Display.setTarget

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `target`

The following specific keys are possible in the return object:

- `targetId`

For details and an example, please see the documentation of the DisplaySetTarget() method in the signoPAD API.

8.6.40 Display.setText

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `xPos`
- `yPos`
- `alignment`
- `text`

The enumeration `TextAlignment` can be used for the key `alignment`.

The following specific keys are possible in the return object:

- `textWidth`

For details and an example, please see the documentation of the DisplaySetText() method in the signoPAD API.

8.6.41 Display.setTextInRect

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `left`
- `top`
- `width`
- `height`
- `alignment`
- `text`

The enumeration `TextAlignment` can be used for the key `alignment`.

The following setters are available in the parameter object:

- `setOptions`

The following specific keys are possible in the return object:

- `fontSizeOrTextHeight`

For details and an example, please see the documentation of the DisplaySetTextInRect() method in the signoPAD API.

8.6.42 Display.setImage

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `xPos`
- `yPos`
- `image`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DisplaySetImage() method in the signoPAD API.

8.6.43 Display.setImageFromStore

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `storeId`

The following specific keys are possible in the return object:

- `storeId`

For details and an example, please see the documentation of the DisplaySetImageFromStore() method in the signoPAD API.

8.6.44 Display.setOverlayRect

Available from server version 1.5.0.0 onwards.

The following keys are mandatory in the parameter object:

- `left`
- `top`
- `width`
- `height`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DisplaySetOverlayRect() method in the signoPAD API.

8.6.45 Display.setScrollPos

Available from server version 1.5.0.0 onwards.

The following keys are mandatory in the parameter object:

- `xPos`
- `yPos`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DisplaySetScrollPos() method in the signoPAD API.

8.6.46 Display.getScrollPos

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `xPos`
- `yPos`

For details and an example, please see the documentation of the DisplayGetScrollPos() method in the signoPAD API.

8.6.47 Display.saveImageAsStream

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `fileType`
- `options`

The enumeration `FileType` can be used for the key `fileType`.

The following specific keys are possible in the return object:

- `image`

For details and an example, please see the documentation of the DisplaySaveImageAsStream() method in the signoPAD API.

8.6.48 Display.setStandbyImage

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `bitmap`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DisplaySetStandbyImage() method in the signoPAD API.

8.6.49 Display.configSlideShowEx

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `slideList`
- `durationList`

The following specific keys are possible in the return object:

- `imageCount`

For details and an example, please see the documentation of the DisplayConfigSlideShowEx() method in the signoPAD API.

8.6.50 Display.getStandbyId

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `numberOfStorages`
- `id`

For details and an example, please see the documentation of the DisplayGetStandbyID() method in the signoPAD API.

8.6.51 Display.getWidth

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `width`

For details and an example, please see the documentation of the DisplayWidth property in the signoPAD API.

8.6.52 Display.getHeight

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `height`

For details and an example, please see the documentation of the DisplayHeight property in the signoPAD API.

8.6.53 Display.getResolution

Available from server version 1.6.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `resolution`

For details and an example, please see the documentation of the DisplayResolution property in signoPAD API.

8.6.54 Display.getTargetWidth

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `targetWidth`

For details and an example, please see the documentation of the DisplayTargetWidth property in the signoPAD API.

8.6.55 Display.getTargetHeight

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `targetHeight`

For details and an example, please see the documentation of the DisplayTargetHeight property in the signoPAD API.

8.6.56 Display.getScrollSpeed

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `speed`

For details and an example, please see the documentation of the DisplayScrollSpeed property in the signoPAD API.

8.6.57 Display.setScrollSpeed

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `speed`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DisplayScrollSpeed property in the signoPAD API.

8.6.58 Display.getRotation

Available from server version 1.1.0.0 onwards.

The function does not expect a parameter object.

The following specific keys are possible in the return object:

- `rotation`

For details and an example, please see the documentation of the DisplayRotation property in the signoPAD API.

8.6.59 Display.setRotation

Available from server version 1.1.0.0 onwards.

The following keys are mandatory in the parameter object:

- `rotation`

No specific keys are possible in the return object.

For details and an example, please see the documentation of the DisplayRotation property in the signoPAD API.

9 Integration at protocol level

Integration at protocol level is more complex than via the JavaScript libraries and should normally only be carried out if no JavaScript library can be used for technical reasons.

Depending on the setup configuration, (see the 'Installation options' and 'Add-on for Internet Explorer' sections), the WebSocket Pad Server can be addressed at `"local.signotecwebsocket.de"`, `"localhost"` or `"127.0.0.1"` on port `"49494"` (by default) and the ActiveX Pad Server via the server name `"signotec"` and the type name `"STPadActiveXServer"`, for example with JavaScript, as follows:

```
var wsUri = "wss://local.signotecwebsocket.de:49494";
// var wsUri = "wss://localhost:49494";
// var wsUri = "wss://127.0.0.1:49494";
// var wsUri = "ws://127.0.0.1:49494";
try {
    stPadServer = new ActiveXObject("signotec.STPadActiveXServer");
    stPadServer.onmessage = onMessage;
} catch (e) {
    stPadServer = new WebSocket(wsUri);
    stPadServer.onmessage = onMessage;
    stPadServer.onopen = onOpen;
    stPadServer.onclose = onClose;
    stPadServer.onerror = onError;
}
```

The ActiveX Pad Server only has the property `onmessage` and the method `send()`; no other WebSocket object properties and methods are required here.

Communication is bidirectional and asynchronous. There are some instructions that expect a response and some instructions that do not expect a response. Each instruction is a JSON string that contains an object with different key-value pairs.

In API mode, all values of the `"TOKEN_CMD"` key begin with `"TOKEN_CMD_API_"` and then contain the method name, for example `"TOKEN_CMD_API_DEVICE_GET_COUNT"`, in order to search for connected devices with the DeviceGetCount() method from signoPAD API. All other keys apply to standard mode.

9.1 Types (TOKEN_TYPE key)

Available from server version 1.0.0.0 onwards.

Each instruction must contain a `"TOKEN_TYPE"` key that defines the type of instruction.

9.1.1 Requests (TOKEN_TYPE_REQUEST value)

Available from server version 1.0.0.0 onwards.

The `"TOKEN_TYPE_REQUEST"` value defines the instruction as a request from the web application to the signoPAD API/Web. The request must contain a command and may contain one or more parameters. A request is always answered by an instruction of type `"TOKEN_TYPE_RESPONSE"`. The web application should always wait for the response before it sends a new request.

Commands for standard mode can be combined with commands for API mode, for server versions 3.1.0.0 or later, provided the connection is opened in API mode (command

"TOKEN_CMD_API_DEVICE_OPEN"). However, this can lead to unexpected behaviour and should therefore only be done in exceptional cases.

The following commands are currently defined for both modes:

- "TOKEN_CMD_GET_INTERFACE_VERSION"
- "TOKEN_CMD_SET_INTERFACE_VERSION"
- "TOKEN_CMD_GET_SERVER_VERSION"

The following commands are currently defined for the standard mode:

- "TOKEN_CMD_SEARCH_FOR_PADS"
- "TOKEN_CMD_OPEN_PAD"
- "TOKEN_CMD_SELECTION_DIALOG"
- "TOKEN_CMD_SIGNATURE_START"
- "TOKEN_CMD_SIGNATURE_RETRY"
- "TOKEN_CMD_SIGNATURE_STOP"
- "TOKEN_CMD_SIGNATURE_CONFIRM"
- "TOKEN_CMD_SIGNATURE_CANCEL"
- "TOKEN_CMD_SIGNATURE_SIGN_DATA"
- "TOKEN_CMD_SIGNATURE_IMAGE"
- "TOKEN_CMD_SIGNING_CERT"
- "TOKEN_CMD_CLOSE_PAD"

The following commands are currently defined for the API mode:

- "TOKEN_CMD_API_DEVICE_SET_COM_PORT"
- "TOKEN_CMD_API_DEVICE_GET_CONNECTION_TYPE"
- "TOKEN_CMD_API_DEVICE_GET_COM_PORT"
- "TOKEN_CMD_API_DEVICE_GET_IP_ADDRESS"
- "TOKEN_CMD_API_DEVICE_GET_COUNT"
- "TOKEN_CMD_API_DEVICE_GET_INFO"
- "TOKEN_CMD_API_DEVICE_GET_VERSION"
- "TOKEN_CMD_API_DEVICE_GET_CAPABILITIES"
- "TOKEN_CMD_API_DEVICE_OPEN"
- "TOKEN_CMD_API_DEVICE_CLOSE"
- "TOKEN_CMD_API_SENSOR_GET_SAMPLE_RATE_MODE"
- "TOKEN_CMD_API_SENSOR_SET_SAMPLE_RATE_MODE"
- "TOKEN_CMD_API_SENSOR_SET_SIGN_RECT"
- "TOKEN_CMD_API_SENSOR_CLEAR_SIGN_RECT"
- "TOKEN_CMD_API_SENSOR_SET_SCROLL_AREA"
- "TOKEN_CMD_API_SENSOR_SET_PEN_SCROLLING_ENABLED"
- "TOKEN_CMD_API_SENSOR_ADD_HOT_SPOT"
- "TOKEN_CMD_API_SENSOR_ADD_SCROLL_HOT_SPOT"
- "TOKEN_CMD_API_SENSOR_SET_HOT_SPOT_MODE"
- "TOKEN_CMD_API_SENSOR_CLEAR_HOT_SPOTS"
- "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION"
- "TOKEN_CMD_API_SIGNATURE_START"
- "TOKEN_CMD_API_SIGNATURE_STOP"
- "TOKEN_CMD_API_SIGNATURE_CONFIRM"
- "TOKEN_CMD_API_SIGNATURE_RETRY"
- "TOKEN_CMD_API_SIGNATURE_CANCEL"
- "TOKEN_CMD_API_SIGNATURE_GET_SIGN_DATA"

- "TOKEN_CMD_API_SIGNATURE_SAVE_AS_STREAM_EX"
- "TOKEN_CMD_API_SIGNATURE_GET_BOUNDS"
- "TOKEN_CMD_API_SIGNATURE_GET_STATE"
- "TOKEN_CMD_API_DISPLAY_ERASE"
- "TOKEN_CMD_API_DISPLAY_ERASE_RECT"
- "TOKEN_CMD_API_DISPLAY_CONFIG_PEN"
- "TOKEN_CMD_API_DISPLAY_SET_FONT"
- "TOKEN_CMD_API_DISPLAY_SET_FONT_COLOR"
- "TOKEN_CMD_API_DISPLAY_SET_TARGET"
- "TOKEN_CMD_API_DISPLAY_SET_TEXT"
- "TOKEN_CMD_API_DISPLAY_SET_TEXT_IN_RECT"
- "TOKEN_CMD_API_DISPLAY_SET_IMAGE"
- "TOKEN_CMD_API_DISPLAY_SET_IMAGE_FROM_STORE"
- "TOKEN_CMD_API_DISPLAY_SET_OVERLAY_RECT"
- "TOKEN_CMD_API_DISPLAY_SET_SCROLL_POS"
- "TOKEN_CMD_API_DISPLAY_GET_SCROLL_POS"
- "TOKEN_CMD_API_DISPLAY_SAVE_IMAGE_AS_STREAM"
- "TOKEN_CMD_API_DISPLAY_SET_STANDBY_IMAGE"
- "TOKEN_CMD_API_DISPLAY_CONFIG_SLIDE_SHOW_EX"
- "TOKEN_CMD_API_DISPLAY_GET_STANDBY_ID"
- "TOKEN_CMD_API_DISPLAY_GET_WIDTH"
- "TOKEN_CMD_API_DISPLAY_GET_HEIGHT"
- "TOKEN_CMD_API_DISPLAY_GET_RESOLUTION"
- "TOKEN_CMD_API_DISPLAY_GET_TARGET_WIDTH"
- "TOKEN_CMD_API_DISPLAY_GET_TARGET_HEIGHT"
- "TOKEN_CMD_API_DISPLAY_GET_SCROLL_SPEED"
- "TOKEN_CMD_API_DISPLAY_SET_SCROLL_SPEED"
- "TOKEN_CMD_API_DISPLAY_GET_ROTATION"
- "TOKEN_CMD_API_DISPLAY_SET_ROTATION"

9.1.2 Responses (TOKEN_TYPE_RESPONSE value)

Available from server version 1.0.0.0 onwards.

The "TOKEN_TYPE_RESPONSE" value defines an instruction as a response from the signoPAD API/Web to a request of type "TOKEN_TYPE_REQUEST" made by the web application. A response always contains a command and one or more parameters. The "TOKEN_CMD_ORIGIN" key always corresponds to the value of the "TOKEN_CMD" key from the request, see also "TOKEN_TYPE_REQUEST".

9.1.3 Events (TOKEN_TYPE_SEND value)

Available from server version 1.0.0.0 onwards.

The "TOKEN_TYPE_SEND" value defines an instruction as an event that is triggered by the server. An event always contains a command and may contain one or more parameters. An event does not expect a response.

The following events are currently defined for both modes:

- "TOKEN_CMD_INCORRECT_COMMAND"
- "TOKEN_CMD_SIGNATURE_POINT"

- "TOKEN_CMD_DISCONNECT"

The following events are currently defined for the standard mode:

- "TOKEN_CMD_SELECTION_CHANGE"
- "TOKEN_CMD_SELECTION_CONFIRM"
- "TOKEN_CMD_SELECTION_CANCEL"
- "TOKEN_CMD_SIGNATURE_RETRY"
- "TOKEN_CMD_SIGNATURE_CONFIRM"
- "TOKEN_CMD_SIGNATURE_CANCEL"
- "TOKEN_CMD_ERROR"

The following events are currently defined for the API mode:

- "TOKEN_CMD_API_SENSOR_HOT_SPOT_PRESSED"
- "TOKEN_CMD_API_DISPLAY_SCROLL_POS_CHANGED"

9.2 Commands (TOKEN_CMD and TOKEN_CMD_ORIGIN keys)

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_CMD" key defines the command that should be executed for requests and events. The value of the "TOKEN_CMD_ORIGIN" key in instructions of type "TOKEN_TYPE_RESPONSE" always corresponds to the value of the "TOKEN_CMD" key from the request.

9.2.1 TOKEN_CMD_GET_INTERFACE_VERSION value

Available from server version 1.0.2.4 onwards.

A request with the command "TOKEN_CMD_GET_INTERFACE_VERSION" returns the set version of the server interface.

As a request, this value has no parameters.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"
- "TOKEN_PARAM_VERSION"

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_GET_INTERFACE_VERSION"}
```

Response example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_GET_INTERFACE_VERSION", "TOKEN_PARAM_RETURN_CODE": "0",
"TOKEN_PARAM_VERSION": "1.0.2.4"}
```

9.2.2 TOKEN_CMD_SET_INTERFACE_VERSION value

Available from server version 1.0.2.4 onwards.

A request with the "TOKEN_CMD_SET_INTERFACE_VERSION" command sets the version of the server interface to be used. The version from which a key or its individual parameters are available is documented for each key. It is recommended that you always set the server version for which development is taking place, in order to ensure that the application functions correctly when a newer version of the signoPad API/Web is installed on the client at a later point.

For example, server versions 1.0.0.0 onwards support version 1.0.0.0 of the server interface, while server versions 1.0.2.4 onwards support version 1.0.2.4 of the server interface.

This value has the following keys as a parameter:

- "TOKEN_PARAM_VERSION"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_SET_INTERFACE_VERSION", "TOKEN_PARAM_VERSION": "1.0.2.4"}
```

9.2.3 TOKEN_CMD_GET_SERVER_VERSION value

Available from server version 1.0.2.4 onwards.

A request with the "TOKEN_CMD_GET_SERVER_VERSION" command delivers the server version. This information can be used to make the behaviour of the web application dependent on the server version available.

As a request, this value has no parameters.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"
- "TOKEN_PARAM_VERSION"

The following keys are only available as of server version 1.6.0.0:

- "TOKEN_PARAM_OS"

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_GET_SERVER_VERSION"}
```

Response example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_GET_SERVER_VERSION", "TOKEN_PARAM_RETURN_CODE": "0",
"TOKEN_PARAM_VERSION": "1.6.0.0", "TOKEN_PARAM_OS": "Windows"}
```

9.2.4 TOKEN_CMD_SEARCH_FOR_PADS value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_SEARCH_FOR_PADS" command starts a search for connected signature devices. If devices are found, their properties are determined and returned in the response.

In Linux, only HID pads are found.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_PAD_SUBSET"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"
- "TOKEN_PARAM_CONNECTED_PADS"

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_SEARCH_FOR_PADS", "TOKEN_PARAM_PAD_SUBSET":
"HID;1;4;IP=192.168.100.100:1002"}
```

9.2.5 TOKEN_CMD_OPEN_PAD value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_OPEN_PAD" command opens a connection to a device.

As a request, this value must have the following keys as a parameter:

- "TOKEN_PARAM_PAD_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"
- "TOKEN_PARAM_PAD_DISPLAY_WIDTH"
- "TOKEN_PARAM_PAD_DISPLAY_HEIGHT"
- "TOKEN_PARAM_PAD_SAMPLING_RATE"

The following keys are only available as of server version 1.0.2.4:

- "TOKEN_PARAM_PAD_X_RESOLUTION"
- "TOKEN_PARAM_PAD_Y_RESOLUTION"

The following keys are only available as of server version 1.6.0.0:

- "TOKEN_PARAM_PAD_DIALOG_WIDTH"
- "TOKEN_PARAM_PAD_DIALOG_HEIGHT"
- "TOKEN_PARAM_PAD_DISPLAY_RESOLUTION"

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD": "TOKEN_CMD_OPEN_PAD",
"TOKEN_PARAM_PAD_INDEX": "0"}
```

Example of the response from server version 1.0.0.0 onwards:

```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_OPEN_PAD", "TOKEN_PARAM_RETURN_CODE": "0",
"TOKEN_PARAM_PAD_DISPLAY_WIDTH": "640", "TOKEN_PARAM_PAD_DISPLAY_HEIGHT":
"480", "TOKEN_PARAM_PAD_SAMPLING_RATE": "250"}
```

Example of the response from server version 1.6.0.0 onwards:

```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_OPEN_PAD", "TOKEN_PARAM_RETURN_CODE": "0",
"TOKEN_PARAM_PAD_DISPLAY_WIDTH": "640", "TOKEN_PARAM_PAD_X_RESOLUTION":
"1920", "TOKEN_PARAM_PAD_DISPLAY_HEIGHT": "480",
"TOKEN_PARAM_PAD_Y_RESOLUTION": "1440", "TOKEN_PARAM_PAD_SAMPLING_RATE":
"250", TOKEN_PARAM_PAD_DIALOG_WIDTH: "640",
TOKEN_PARAM_PAD_DIALOG_HEIGHT: "424", TOKEN_PARAM_PAD_DISPLAY_RESOLUTION:
"160.04463191221529"}
```

9.2.6 TOKEN_CMD_SELECTION_DIALOG value

Available from server version 1.4.0.0 onwards.

A request with the `"TOKEN_CMD_SELECTION_DIALOG"` command displays a dialog with freely configurable selection fields, as well as 'Cancel' and 'Confirm' buttons, provided that a connection has been opened beforehand using the `"TOKEN_CMD_OPEN_PAD"` command. The selection fields are dynamically positioned, depending on the text length and size of the signature device screen, and divided into several pages if necessary. If the value of a selection field is changed by a user by clicking it, an event of type `"TOKEN_CMD_SELECTION_CHANGE"` is triggered. If the 'Cancel' button is activated, an event of type `"TOKEN_CMD_SELECTION_CANCEL"` is triggered. If the 'Confirm' button is activated on the last page, an event of type `"TOKEN_CMD_SELECTION_CONFIRM"` is triggered. If the 'Confirm' button is activated on a previous page, the next page is displayed without informing the app. If an error occurs during this process, an event of type `"TOKEN_CMD_ERROR"` is triggered.

The request with the `"TOKEN_CMD_SELECTION_DIALOG"` command is not implemented under Linux and returns the error -25.

As a request, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_FIELD_LIST"`

As a response, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_RETURN_CODE"`
- `"TOKEN_PARAM_ERROR_DESCRIPTION"`

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_SELECTION_DIALOG", "TOKEN_PARAM_FIELD_LIST":
[{"TOKEN_PARAM_FIELD_ID": "Check1", "TOKEN_PARAM_FIELD_TEXT": "I agree to
the processing of my data.", "TOKEN_PARAM_FIELD_CHECKED": "FALSE",
"TOKEN_PARAM_FIELD_REQUIRED": "TRUE"}, {"TOKEN_PARAM_FIELD_ID": "Check2",
"TOKEN_PARAM_FIELD_TEXT": "I would like to subscribe to the newsletter.",
"TOKEN_PARAM_FIELD_CHECKED": "TRUE", "TOKEN_PARAM_FIELD_REQUIRED":
"FALSE"}]}
```

9.2.7 TOKEN_CMD_SELECTION_CHANGE value

Available from server version 1.4.0.0 onwards.

An event with the `"TOKEN_CMD_SELECTION_CHANGE"` command will be triggered if the user clicks a field of the selection dialog on the signature device with the pen.

This value can have the following keys as a parameter:

- `"TOKEN_PARAM_FIELD_ID"`
- `"TOKEN_PARAM_FIELD_CHECKED"`

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_SEND", "TOKEN_CMD":
"TOKEN_CMD_SELECTION_CHANGE", "TOKEN_PARAM_FIELD_ID": "Check1",
"TOKEN_PARAM_FIELD_CHECKED": "TRUE"}
```

9.2.8 TOKEN_CMD_SELECTION_CONFIRM value

Available from server version 1.4.0.0 onwards.

An event with the `"TOKEN_CMD_SELECTION_CONFIRM"` command will be triggered if the user clicks the 'Confirm' button on the last page of the selection dialog on the signature device with the pen. The typical reaction to this event should be a request with the `"TOKEN_CMD_SIGNATURE_START"` or `"TOKEN_CMD_CLOSE_PAD"` command.

This value does not have any parameters.

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_SEND", "TOKEN_CMD":
"TOKEN_CMD_SELECTION_CONFIRM"}
```

9.2.9 TOKEN_CMD_SELECTION_CANCEL value

Available from server version 1.4.0.0 onwards.

An event with the `"TOKEN_CMD_SELECTION_CANCEL"` command will be triggered if the user clicks the 'Cancel' button of a selection dialog on the signature device with the pen. The typical reaction to this event should be a request with the `"TOKEN_CMD_CLOSE_PAD"` command.

This value does not have any parameters.

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_SEND", "TOKEN_CMD":
"TOKEN_CMD_SELECTION_CANCEL"}
```

9.2.10 TOKEN_CMD_SIGNATURE_START value

Available from server version 1.0.0.0 onwards.

A request with the `"TOKEN_CMD_SIGNATURE_START"` command starts the signature capture process, provided a connection has been opened beforehand using the `"TOKEN_CMD_OPEN_PAD"` command.

As a request, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_FIELD_NAME"`
- `"TOKEN_PARAM_CUSTOM_TEXT"`

The following keys are only available in Windows and only as of server version 1.3.0.0:

- `"TOKEN_PARAM_PAD_ENCRYPTION"`
- `"TOKEN_PARAM_DOCHASH"`
- `"TOKEN_PARAM_ENCRYPTION_CERT"`

The following keys are only available in Windows and only as of server version 1.6.0.0:

- `"TOKEN_PARAM_PAD_DIALOG_IMAGE"`

The following keys are only available in Windows and only as of server version 1.7.0.0:

- `"TOKEN_PARAM_TEXT_LAYOUT"`

The following keys are only available in Windows and only as of server version 3.2.0.0:

- `"TOKEN_PARAM_ENCRYPTION_CERT_ONLY_WHEN_EMPTY"`

As a response, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_RETURN_CODE"`
- `"TOKEN_PARAM_ERROR_DESCRIPTION"`

Examples:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_SIGNATURE_START", "TOKEN_PARAM_FIELD_NAME": "Customer
Signature", "TOKEN_PARAM_CUSTOM_TEXT": "Please sign here!"}
```


```
{"TOKEN_TYPE":"TOKEN_TYPE_REQUEST",
"TOKEN_CMD":"TOKEN_CMD_SIGNATURE_START", "TOKEN_PARAM_PAD_DIALOG_IMAGE":
"iVBORw0KGgoAAAANSUhEUgAAAAEAAAABCAIAAACQd1PeAAAAAXNSR0IArs4c6QAA
AARnQU1BAACxjwv8YQUAAAAJcEhZcwAADsMAAA7DAcdvqGQAAAAMSURBVBhXY/j/
/z8ABf4C/qc1gYQAAAAASUVORK5CYII=", "TOKEN_PARAM_PAD_ENCRYPTION":"TRUE",
"TOKEN_PARAM_DOCHASH":"AAECAwQFBgcICQoLDA0ODxAREhMUFRYXGBkaGxwdHh8=",
"TOKEN_PARAM_ENCRYPTION_CERT":"MIICqTCCAZGgAwIBAgIBATANBgkqhkiXy1bpSA==",
"TOKEN_PARAM_ENCRYPTION_CERT_ONLY_WHEN_EMPTY":"TRUE"}
```

```
{"TOKEN_TYPE":"TOKEN_TYPE_REQUEST",
"TOKEN_CMD":"TOKEN_CMD_SIGNATURE_START",
"TOKEN_PARAM_TEXT_LAYOUT":{"TOKEN_PARAM_LAYOUT_ID": "0",
"TOKEN_PARAM_TEXT_BLOCKS": [{"TOKEN_PARAM_TEXT": "I confirm this
agreement.", "TOKEN_PARAM_WIDTH": "100", "TOKEN_PARAM_HEIGHT": "60",
"TOKEN_PARAM_MAX_FONT_SIZE": "100", "TOKEN_PARAM_CONSTANT": "TRUE"},
{"TOKEN_PARAM_WIDTH": "100", "TOKEN_PARAM_HEIGHT": "7"},
{"TOKEN_PARAM_TEXT": "Account:", "TOKEN_PARAM_WIDTH": "32",
"TOKEN_PARAM_HEIGHT": "17", "TOKEN_PARAM_FONT_STYLE": ["UNDERLINE"],
"TOKEN_PARAM_MAX_FONT_SIZE": "100", "TOKEN_PARAM_FONT_SIZE_ID" : "0"},
{"TOKEN_PARAM_WIDTH": "2", "TOKEN_PARAM_HEIGHT": "17"},
{"TOKEN_PARAM_TEXT": "Agreement ID:", "TOKEN_PARAM_WIDTH": "32",
"TOKEN_PARAM_HEIGHT": "17", "TOKEN_PARAM_FONT_STYLE": ["UNDERLINE"],
"TOKEN_PARAM_MAX_FONT_SIZE": "100", "TOKEN_PARAM_FONT_SIZE_ID" : "0"},
{"TOKEN_PARAM_WIDTH": "2", "TOKEN_PARAM_HEIGHT": "17"},
{"TOKEN_PARAM_TEXT": "Name:", "TOKEN_PARAM_WIDTH": "32",
"TOKEN_PARAM_HEIGHT": "17", "TOKEN_PARAM_FONT_STYLE": ["UNDERLINE"],
"TOKEN_PARAM_MAX_FONT_SIZE": "100", "TOKEN_PARAM_FONT_SIZE_ID" : "0"},
{"TOKEN_PARAM_TEXT": "123456", "TOKEN_PARAM_WIDTH": "32",
"TOKEN_PARAM_HEIGHT": "12", "TOKEN_PARAM_MAX_FONT_SIZE": "100",
"TOKEN_PARAM_FONT_SIZE_ID" : "1"}, {"TOKEN_PARAM_WIDTH": "2",
"TOKEN_PARAM_HEIGHT": "16"}, {"TOKEN_PARAM_TEXT": "7890",
"TOKEN_PARAM_WIDTH": "32", "TOKEN_PARAM_HEIGHT": "16",
"TOKEN_PARAM_MAX_FONT_SIZE": "100", "TOKEN_PARAM_FONT_SIZE_ID" : "1"},
{"TOKEN_PARAM_WIDTH": "2", "TOKEN_PARAM_HEIGHT": "16"},
{"TOKEN_PARAM_TEXT": "John Doe", "TOKEN_PARAM_WIDTH": "32",
"TOKEN_PARAM_HEIGHT": "16", "TOKEN_PARAM_MAX_FONT_SIZE": "100",
"TOKEN_PARAM_FONT_SIZE_ID" : "1"}]}}
```

9.2.11 TOKEN_CMD_SIGNATURE_RETRY value

Available from server version 1.0.0.0 onwards.

A request with the `"TOKEN_CMD_SIGNATURE_RETRY"` command discards the signature data without ending the signature capture process and deletes the rendered signature from the signature device's display. This command will start a new capture process if the signature capture process was terminated beforehand using the `"TOKEN_CMD_SIGNATURE_STOP"` command.

An event with the `"TOKEN_CMD_SIGNATURE_RETRY"` command will be triggered if the user clicks the 'Retry' button on the signature device with the pen. The typical reaction to this event should be a request with the `"TOKEN_CMD_SIGNATURE_RETRY"` command.

This value has no parameters either as a request or as an event.

As a response, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_RETURN_CODE"`
- `"TOKEN_PARAM_ERROR_DESCRIPTION"`

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_SIGNATURE_RETRY"}
```

9.2.12 TOKEN_CMD_SIGNATURE_STOP value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_SIGNATURE_STOP" command terminates the signature capture process that is currently running and caches the captured signature data.

As a request, this value has no parameters.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_SIGNATURE_STOP"}
```

9.2.13 TOKEN_CMD_SIGNATURE_CONFIRM value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_SIGNATURE_CONFIRM" command terminates the signature capture process that is currently running, caches the captured signature data and, in contrast to "TOKEN_CMD_SIGNATURE_STOP", erases the entire device LCD.

An event with the "TOKEN_CMD_SIGNATURE_CONFIRM" command will be triggered if the user clicks the 'Confirm' button on the signature device with the pen. The typical reaction to this event should be a request with the "TOKEN_CMD_SIGNATURE_STOP" or "TOKEN_CMD_SIGNATURE_CONFIRM" command.

This value has no parameters either as a request or as an event.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

The value of the "TOKEN_PARAM_RETURN_CODE" key when the "TOKEN_CMD_SIGNATURE_CONFIRM" command has been successfully called contains the number of captured signature points.

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_SIGNATURE_CONFIRM"}
```

Response example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_SIGNATURE_CONFIRM", "TOKEN_PARAM_RETURN_CODE": "276"}
```

9.2.14 TOKEN_CMD_SIGNATURE_CANCEL value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_SIGNATURE_CANCEL" command terminates the signature capture process that is currently running, discards the signature data and erases the entire device LCD.

An event with the "TOKEN_CMD_SIGNATURE_CANCEL" command will be triggered if the user clicks the 'Cancel' button on the signature device with the pen. The typical reaction to this event should be a request with the "TOKEN_CMD_SIGNATURE_CANCEL" command.

This value has no parameters either as a request or as an event.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_SIGNATURE_CANCEL"}
```

9.2.15 TOKEN_CMD_SIGNATURE_SIGN_DATA value

Available from server version 1.0.0.0 onwards.

A request with the "TOKEN_CMD_SIGNATURE_SIGN_DATA" command retrieves the SignData of a captured signature from the device. SignData is an encrypted, compressed, biometric data format that can be stored in a database or as a tag in a TIFF or PDF document.

If the command "TOKEN_CMD_SIGNATURE_START" is started with the parameter "TOKEN_PARAM_PAD_ENCRYPTION" = "TRUE" ('RSA in pad' mode), the SignData encrypted on the pad and the response contains the parameter "TOKEN_PARAM_CERT_ID".

As a request, this value can have the following keys as a parameter, for server versions 3.2.0.0 or later:

- "TOKEN_PARAM_SIGNATURE_RSA_SCHEME"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"
- "TOKEN_PARAM_SIGNATURE_SIGN_DATA"

The following keys are only available as of server version 1.3.0.0:

- "TOKEN_PARAM_CERT_ID"

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_SIGNATURE_SIGN_DATA", "TOKEN_PARAM_SIGNATURE_RSA_SCHEME":
"PSS"}
```

Response example:


```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_SIGNATURE_SIGN_DATA", "TOKEN_PARAM_RETURN_CODE": "0",
"TOKEN_PARAM_SIGNATURE_SIGN_DATA": "CwAAAAEAAABmABACwgjCCAAgABu=",
"TOKEN_PARAM_CERT_ID": "O=Demo signotec, SNR=01",
"TOKEN_PARAM_SIGNATURE_RSA_SIGNATURE":
"Am7q+LWbqgVE5pM2eV8kaLe5bGW+KKB9i2kPG6LDOXRTNzeRLmEXvHg/EFCYub9LcFUrTGpZ
yTbKriD8LygMhpskki9yIOQGJVANx3P1xplqp24aexbrRNdA5FUEh8ApOs6ihjPET8WiqV71S
vK/E2jk+oTdPDbLle/xFg2WEyI="}
```


9.2.16 TOKEN_CMD_SIGNATURE_IMAGE value

Available from server version 1.0.0.0 onwards.

A request with the `"TOKEN_CMD_SIGNATURE_IMAGE"` command returns a captured signature as image data. The image is created with the aspect ratio of the rectangle that surrounds the signature.

The request with the `"TOKEN_CMD_SIGNATURE_IMAGE"` command is not implemented under Linux and returns the error -25.

As a request, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_FILE_TYPE"`

The following keys are only available as of server version 1.1.0.0:

- `"TOKEN_PARAM_PEN_WIDTH"`

As a response, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_RETURN_CODE"`
- `"TOKEN_PARAM_ERROR_DESCRIPTION"`
- `"TOKEN_PARAM_FILE"`

Example:


```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_SIGNATURE_IMAGE", "TOKEN_PARAM_FILE_TYPE": "1",
"TOKEN_PARAM_PEN_WIDTH": "0"}
```


9.2.17 TOKEN_CMD_SIGNING_CERT value

Available from server version 1.7.0.0 onwards.

A request with the command `"TOKEN_CMD_SIGNING_CERT"` retrieves the signature certificate stored in the signature device.

As a request, this value has no parameters.

As a response, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_RETURN_CODE"`
- `"TOKEN_PARAM_ERROR_DESCRIPTION"`
- `"TOKEN_PARAM_SIGNING_CERT"`

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_SIGNING_CERT"}
```

9.2.18 TOKEN_CMD_CLOSE_PAD value

Available from server version 1.0.0.0 onwards.

A request with the `"TOKEN_CMD_CLOSE_PAD"` command closes the connection to a device. Before closing, a currently running signature capture process is terminated. Captured signature data is discarded.

As a request, this value must have the following keys as a parameter:

- `"TOKEN_PARAM_PAD_INDEX"`

As a response, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_RETURN_CODE"`
- `"TOKEN_PARAM_ERROR_DESCRIPTION"`

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD": "TOKEN_CMD_CLOSE_PAD",
"TOKEN_PARAM_PAD_INDEX": "0"}
```

9.2.19 TOKEN_CMD_INCORRECT_COMMAND value

Available from server version 3.1.0.0 onwards.

An event of type `"TOKEN_CMD_INCORRECT_COMMAND"` is triggered if a command could not be interpreted because it is incorrect.

This value has the following keys as a parameter:

- `"TOKEN_PARAM_EXCEPTION_CAUSE"`

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_SEND", "TOKEN_CMD":
"TOKEN_CMD_INCORRECT_COMMAND", "TOKEN_PARAM_EXCEPTION_CAUSE": "(1):
expected object or array"}
```

9.2.20 TOKEN_CMD_SIGNATURE_POINT value

Available from server version 1.0.0.0 onwards.

An event of type `"TOKEN_CMD_SIGNATURE_POINT"` is triggered for each captured point during the signing process. The web application can use this event to render the signature in real time.

This value has the following keys as a parameter:

- `"TOKEN_PARAM_POINT"`

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_SEND", "TOKEN_CMD":
"TOKEN_CMD_SIGNATURE_POINT", "TOKEN_PARAM_POINT": {"x": "218", "y":
"245", "p": "0"}}
```

9.2.21 TOKEN_CMD_DISCONNECT value

Available from server version 1.0.0.0 onwards.

An event of type `"TOKEN_CMD_DISCONNECT"` is triggered as soon as the connection to a device is terminated via an external event (for example, the device is unplugged). The event will only be triggered for a device that has been opened beforehand using a request with the `"TOKEN_CMD_OPEN_PAD"` or `"TOKEN_CMD_API_DEVICE_OPEN"` command.

This value has the following keys as a parameter:

- `"TOKEN_PARAM_PAD_INDEX"`

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_SEND", "TOKEN_CMD": "TOKEN_CMD_DISCONNECT", "
TOKEN_PARAM_PAD_INDEX": "0"}
```

9.2.22 TOKEN_CMD_ERROR value

Available from server version 1.4.0.0 onwards.

An event with the `"TOKEN_CMD_ERROR"` command is triggered if an error occurs outside of a request.

This value can have the following keys as a parameter:

- `"TOKEN_PARAM_ERROR_CONTEXT"`
- `"TOKEN_PARAM_RETURN_CODE"`
- `"TOKEN_PARAM_ERROR_DESCRIPTION"`

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_SEND", "TOKEN_CMD": "TOKEN_CMD_ERROR",
"TOKEN_PARAM_ERROR_CONTEXT": "TOKEN_CMD_SELECTION_DIALOG",
"TOKEN_PARAM_RETURN_CODE": "-400", "TOKEN_PARAM_ERROR_DESCRIPTION": "Not
all mandatory fields have been selected."}
```

9.2.23 TOKEN_CMD_API_DEVICE_SET_COM_PORT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_PORT_LIST"`

As a response, this value can have the following keys as a parameter:

- `"TOKEN_PARAM_RETURN_CODE"`
- `"TOKEN_PARAM_ERROR_DESCRIPTION"`

For details and an example, please see the documentation of the DeviceSetComPort() method in the signoPAD API.

9.2.24 TOKEN_CMD_API_DEVICE_GET_CONNECTION_TYPE value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceGetConnectionType() method in the signoPAD API.

9.2.25 TOKEN_CMD_API_DEVICE_GET_COM_PORT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceGetComPort() method in the signoPAD API.

9.2.26 TOKEN_CMD_API_DEVICE_GET_IP_ADDRESS value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_IPADDRESS"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceGetIPAddress() method in the signoPAD API.

9.2.27 TOKEN_CMD_API_DEVICE_GET_COUNT value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceGetCount() method in the signoPAD API.

9.2.28 TOKEN_CMD_API_DEVICE_GET_INFO value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_SERIAL"
- "TOKEN_PARAM_TYPE"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceGetInfo() method in the signoPAD API.

9.2.29 TOKEN_CMD_API_DEVICE_GET_VERSION value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_VERSION"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceGetVersion() method in the signoPAD API.

9.2.30 TOKEN_CMD_API_DEVICE_GET_CAPABILITIES value

Available from server version 1.6.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"

- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceGetCapabilities() method in signoPAD API.

9.2.31 TOKEN_CMD_API_DEVICE_OPEN value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"
- "TOKEN_PARAM_ERASE_DISPLAY"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceOpen() method in the signoPAD API.

9.2.32 TOKEN_CMD_API_DEVICE_CLOSE value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_INDEX"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DeviceClose() method in the signoPAD API.

9.2.33 TOKEN_CMD_API_SENSOR_GET_SAMPLE_RATE_MODE value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SensorGetSampleRateMode() method in the signoPAD API.

9.2.34 TOKEN_CMD_API_SENSOR_SET_SAMPLE_RATE_MODE value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_MODE"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SensorSetSampleRateMode() method in the signoPAD API.

9.2.35 TOKEN_CMD_API_SENSOR_SET_SIGN_RECT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_LEFT"
- "TOKEN_PARAM_TOP"
- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_HEIGHT"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SensorSetSignRect() method in the signoPAD API.

9.2.36 TOKEN_CMD_API_SENSOR_CLEAR_SIGN_RECT value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SensorClearSignRect() method in the signoPAD API.

9.2.37 TOKEN_CMD_API_SENSOR_SET_SCROLL_AREA value

Available from server version 1.5.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_LEFT"
- "TOKEN_PARAM_TOP"
- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_HEIGHT"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SensorSetScrollArea() method in the signoPAD API.

9.2.38 TOKEN_CMD_API_SENSOR_SET_PEN_SCROLLING_ENABLED value

Available from server version 1.5.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_ENABLE"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SensorSetPenScrollingEnabled() method in the signoPAD API.

9.2.39 TOKEN_CMD_API_SENSOR_ADD_HOT_SPOT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_LEFT"
- "TOKEN_PARAM_TOP"
- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_HEIGHT"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SensorAddHotSpot() method in the signoPAD API.

9.2.40 TOKEN_CMD_API_SENSOR_ADD_SCROLL_HOT_SPOT value

Available from server version 1.5.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_LEFT"
- "TOKEN_PARAM_TOP"
- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_HEIGHT"
- "TOKEN_PARAM_TYPE"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SensorAddScrollHotSpot() method in the signoPAD API.

9.2.41 TOKEN_CMD_API_SENSOR_SET_HOT_SPOT_MODE value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_MODE"
- "TOKEN_PARAM_HOTSPOT_ID"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SensorSetHotSpotMode() method in the signoPAD API.

9.2.42 TOKEN_CMD_API_SENSOR_CLEAR_HOT_SPOTS value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SensorClearHotSpots() method in the signoPAD API.

9.2.43 TOKEN_CMD_API_SENSOR_HOT_SPOT_PRESSED value

Available from server version 1.1.0.0 onwards.

This value has the following keys as a parameter:

- "TOKEN_PARAM_HOTSPOT_ID"

For details and an example, please see the documentation of the SensorHotSpotPressed() event in the signoPAD API.

9.2.44 TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION value

Available from server version 1.1.0.0 onwards.

A request with the "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION" command delivers the resolution of the signature data that is delivered in the event of the "TOKEN_CMD_SIGNATURE_POINT" type.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_PAD_X_RESOLUTION"
- "TOKEN_PARAM_PAD_Y_RESOLUTION"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION"}
```

Response example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION", "TOKEN_PARAM_RETURN_CODE": "0",
"TOKEN_PARAM_PAD_X_RESOLUTION": "1920", "TOKEN_PARAM_PAD_Y_RESOLUTION":
"1440"}
```

9.2.45 TOKEN_CMD_API_SIGNATURE_START value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureStart() method in the signoPAD API.

9.2.46 TOKEN_CMD_API_SIGNATURE_STOP value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureStop() method in the signoPAD API.

9.2.47 TOKEN_CMD_API_SIGNATURE_CONFIRM value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureConfirm() method in the signoPAD API.

9.2.48 TOKEN_CMD_API_SIGNATURE_RETRY value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureRetry() method in the signoPAD API.

9.2.49 TOKEN_CMD_API_SIGNATURE_CANCEL value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_ERASE"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureCancel() method in the signoPAD API.

9.2.50 TOKEN_CMD_API_SIGNATURE_GET_SIGN_DATA value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_SIGN_DATA"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureGetSignData() method in the signoPAD API.

9.2.51 TOKEN_CMD_API_SIGNATURE_SAVE_AS_STREAM_EX value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RESOLUTION"
- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_HEIGHT"
- "TOKEN_PARAM_FILE_TYPE"
- "TOKEN_PARAM_PEN_WIDTH"
- "TOKEN_PARAM_PEN_COLOR"
- "TOKEN_PARAM_OPTIONS"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_IMAGE"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureSaveAsStreamEx() method in the signoPAD API.

9.2.52 TOKEN_CMD_API_SIGNATURE_GET_BOUNDS value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_OPTIONS"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_LEFT"
- "TOKEN_PARAM_TOP"
- "TOKEN_PARAM_RIGHT"
- "TOKEN_PARAM_BOTTOM"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureGetBounds() method in the signoPAD API.

9.2.53 TOKEN_CMD_API_SIGNATURE_GET_STATE value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the SignatureState property in the signoPAD API.

9.2.54 TOKEN_CMD_API_DISPLAY_ERASE value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayErase() method in the signoPAD API.

9.2.55 TOKEN_CMD_API_DISPLAY_ERASE_RECT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_LEFT"
- "TOKEN_PARAM_TOP"
- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_HEIGHT"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayEraseRect() method in the signoPAD API.

9.2.56 TOKEN_CMD_API_DISPLAY_CONFIG_PEN value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_PEN_COLOR"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayConfigPen() method in the signoPAD API.

9.2.57 TOKEN_CMD_API_DISPLAY_SET_FONT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_NAME"
- "TOKEN_PARAM_SIZE"
- "TOKEN_PARAM_OPTIONS"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySetFont() method in the signoPAD API.

9.2.58 TOKEN_CMD_API_DISPLAY_SET_FONT_COLOR value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_FONT_COLOR"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySetFontColor() method in the signoPAD API.

9.2.59 TOKEN_CMD_API_DISPLAY_SET_TARGET value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_TARGET"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySetTarget() method in the signoPAD API.

9.2.60 TOKEN_CMD_API_DISPLAY_SET_TEXT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_X_POS"
- "TOKEN_PARAM_Y_POS"
- "TOKEN_PARAM_ALIGNMENT"
- "TOKEN_PARAM_TEXT"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySetText() method in the signoPAD API.

9.2.61 TOKEN_CMD_API_DISPLAY_SET_TEXT_IN_RECT value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_LEFT"
- "TOKEN_PARAM_TOP"
- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_HEIGHT"
- "TOKEN_PARAM_ALIGNMENT"
- "TOKEN_PARAM_TEXT"
- "TOKEN_PARAM_OPTIONS"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySetTextInRect() method in the signoPAD API.

9.2.62 TOKEN_CMD_API_DISPLAY_SET_IMAGE value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_X_POS"
- "TOKEN_PARAM_Y_POS"
- "TOKEN_PARAM_BITMAP"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySetImage() method in the signoPAD API.

9.2.63 TOKEN_CMD_API_DISPLAY_SET_IMAGE_FROM_STORE value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_STORE_ID"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySetImageFromStore() method in the signoPAD API.

9.2.64 TOKEN_CMD_API_DISPLAY_SET_OVERLAY_RECT value

Available from server version 1.5.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_LEFT"
- "TOKEN_PARAM_TOP"
- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_HEIGHT"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySetOverlayRect() method in the signoPAD API.

9.2.65 TOKEN_CMD_API_DISPLAY_SET_SCROLL_POS value

Available from server version 1.5.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_X_POS"
- "TOKEN_PARAM_Y_POS"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySetScrollPos() method in the signoPAD API.

9.2.66 TOKEN_CMD_API_DISPLAY_GET_SCROLL_POS value

Available from server version 1.5.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_X_POS"
- "TOKEN_PARAM_Y_POS"
- "TOKEN_PARAM_RETURN_CODE"

- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayGetScrollPos() method in the signoPAD API.

9.2.67 TOKEN_CMD_API_DISPLAY_SAVE_IMAGE_AS_STREAM value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_FILE_TYPE"
- "TOKEN_PARAM_OPTIONS"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_IMAGE"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySaveImageAsStream() method in the signoPAD API.

9.2.68 TOKEN_CMD_API_DISPLAY_SET_STANDBY_IMAGE value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_BITMAP"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplaySetStandbyImage() method in the signoPAD API.

9.2.69 TOKEN_CMD_API_DISPLAY_CONFIG_SLIDE_SHOW_EX value

Available from server version 1.1.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_SLIDE_LIST"
- "TOKEN_PARAM_DURATION_LIST"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayConfigSlideShowEx() method in the signoPAD API.

9.2.70 TOKEN_CMD_API_DISPLAY_GET_STANDBY_ID value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_ID"
- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayGetStandbyID() method in the signoPAD API.

9.2.71 TOKEN_CMD_API_DISPLAY_GET_WIDTH value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayWidth property in the signoPAD API.

9.2.72 TOKEN_CMD_API_DISPLAY_GET_HEIGHT value

Available from server version 1.1.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayHeight property in the signoPAD API.

9.2.73 TOKEN_CMD_API_DISPLAY_GET_RESOLUTION value

Available from server version 1.6.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayResolution property in signoPAD API.

9.2.74 TOKEN_CMD_API_DISPLAY_GET_TARGET_WIDTH value

Available from server version 1.5.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayTargetWidth property in the signoPAD API.

9.2.75 TOKEN_CMD_API_DISPLAY_GET_TARGET_HEIGHT value

Available from server version 1.5.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayTargetHeight property in the signoPAD API.

9.2.76 TOKEN_CMD_API_DISPLAY_GET_SCROLL_SPEED value

Available from server version 1.5.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayScrollSpeed property in the signoPAD API.

9.2.77 TOKEN_CMD_API_DISPLAY_SET_SCROLL_SPEED value

Available from server version 1.5.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_SPEED"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayScrollSpeed property in the signoPAD API.

9.2.78 TOKEN_CMD_API_DISPLAY_GET_ROTATION value

Available from server version 1.3.0.0 onwards.

As a request, this value has no keys as a parameter.

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayRotation property in the signoPAD API.

9.2.79 TOKEN_CMD_API_DISPLAY_SET_ROTATION value

Available from server version 1.3.0.0 onwards.

As a request, this value can have the following keys as a parameter:

- "TOKEN_PARAM_ROTATION"

As a response, this value can have the following keys as a parameter:

- "TOKEN_PARAM_RETURN_CODE"
- "TOKEN_PARAM_ERROR_DESCRIPTION"

For details and an example, please see the documentation of the DisplayRotation property in the signoPAD API.

9.2.80 TOKEN_CMD_API_DISPLAY_SCROLL_POS_CHANGED value

Available from server version 1.5.0.0 onwards.

This value has the following keys as a parameter:

- "TOKEN_PARAM_X_POS"
- "TOKEN_PARAM_Y_POS"

For details and an example, please see the documentation of the DisplayScrollPosChanged() event in the signoPAD API.

9.3 Parameter

Some instructions may contain parameters. These are defined below.

9.3.1 Special features of parameters in API mode

9.3.1.1 Numbers and enumerations

Numbers must be passed as strings. The point must be used as the decimal separator for floating point numbers. Hexadecimal numbers can be passed by placing `"0x"` in front; they can, of course, also be passed as decimal numbers. Enumerations cannot be used; instead the numerical values must be passed.

Numbers – examples

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_API_DISPLAY_SET_TEXT_IN_RECT", "TOKEN_PARAM_LEFT": "0",
"TOKEN_PARAM_TOP": "0", "TOKEN_PARAM_WIDTH": "0", "TOKEN_PARAM_HEIGHT":
"0", "TOKEN_PARAM_ALIGNMENT": "0", "TOKEN_PARAM_TEXT": "Please sign
here:", "TOKEN_PARAM_OPTIONS": "0x01"}
```

9.3.1.2 Boolean parameters

Boolean parameters must be passed as `"TRUE"` or `"FALSE"`.

Boolean parameter – example

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_API_SENSOR_SET_PEN_SCROLLING_ENABLED", "TOKEN_PARAM_ENABLE":
"TRUE"}
```

9.3.1.3 Colours

Parameters that define colours must be passed as a number representing an RGB value. The last-but-two byte determines the share of the blue channel, the last-but-one byte the share of the green channel and the last byte determines the share of the red channel. Therefore, the colour blue corresponds to `"0xFF0000"` or `"16711680"`, the colour green to `"0x00FF00"` or `"65280"` and the colour red to `"0x0000FF"` or `"255"`.

Colour parameter – example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_API_DISPLAY_CONFIG_PEN", "TOKEN_PARAM_WIDTH": "2",
"TOKEN_PARAM_PEN_COLOR": "0xFF0000"}
```

9.3.1.4 Pointer parameters

Pointers cannot be used. Instead, the data can be passed directly with the request. The data changed by the method is delivered in the response.

Pointer as an in parameter – example

```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_API_DEVICE_SET_COM_PORT", "TOKEN_PARAM_PORT_LIST":
"HID;1;4;IP=192.168.100.100:1002"}
```

Pointer as an out parameter – example

```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_API_DEVICE_GET_INFO", "TOKEN_PARAM_SERIAL": "1500001234",
"TOKEN_PARAM_TYPE": "15"}
```

9.3.1.5 Byte arrays and images

Instead of byte arrays and images, Base64 strings must be passed and returned. The PNG format is recommended for images.

Passing a PNG image – example


```
{"TOKEN_TYPE": "TOKEN_TYPE_REQUEST", "TOKEN_CMD":
"TOKEN_CMD_API_DISPLAY_SET_IMAGE", "TOKEN_PARAM_X_POS": "0",
"TOKEN_PARAM_Y_POS": "0", "TOKEN_PARAM_BITMAP":
"iVBORw0KGgoAAAANSUhEUgAAAAEAAAABCAIAAACQd1PeAAAAAXNSR0IArs4c6QAA
AARnQU1BAACxjwv8YQUAAAAJcEhZcwAADsMAAA7DAcdvqGQAAAAMSURBVBhXY/j/
/z8ABf4C/qc1gYQAAAAASUVORK5CYII="}
```


Return of a byte array – example


```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_API_SIGNATURE_GET_SIGN_DATA", "TOKEN_PARAM_SIGN_DATA":
"CgAAAAEAAABvAAgCVQVVBVUVABAEAAEAAQAABA2IYa1x2GVCVU4WzHpbdUCrrInm
GTLTbzfOOd/WFQo47vAi9Af4AFc="}
```


9.3.1.6 Files

Files (for example, images from the local hard disk) are not supported. Data can only be exchanged as Base64-encoded strings.

9.3.2 TOKEN_PARAM_RETURN_CODE key

Available from server version 1.0.0.0 onwards.

The `"TOKEN_PARAM_RETURN_CODE"` key contains an error number as a value. `"0"` means that no error has occurred. A negative value indicates that an error occurred during the process. A positive value has different meanings depending on the command. If the value is negative, the response can also contain the `"TOKEN_PARAM_ERROR_DESCRIPTION"` key. For details, please refer to section 'Error codes'.

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_SEARCH_FOR_PADS", "TOKEN_PARAM_RETURN_CODE": "0"}
```

9.3.3 TOKEN_PARAM_ERROR_DESCRIPTION key

Available from server version 1.0.0.0 onwards.

The `"TOKEN_PARAM_ERROR_DESCRIPTION"` key contains an error description as a value.

Example:

```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_OPEN_PAD", "TOKEN_PARAM_PAD_INDEX": "1",
"TOKEN_PARAM_RETURN_CODE": "-4", "TOKEN_PARAM_ERROR_DESCRIPTION": "The
signing pad is already being used by another application."}
```

9.3.4 TOKEN_PARAM_ERROR_CONTEXT key

Available from server version 1.4.0.0 onwards.

The `"TOKEN_PARAM_ERROR_CONTEXT"` key contains the command within the context of which the error occurred.

For an example, see the `"TOKEN_CMD_ERROR"` value.

9.3.5 TOKEN_PARAM_PAD_SUBSET key

Available from server version 1.0.0.0 onwards.

The `"TOKEN_PARAM_PAD_SUBSET"` key defines the interfaces to be searched when looking for devices. For details, please refer to section 'List for pad search'.

For an example, see the `"TOKEN_CMD_SEARCH_FOR_PADS"` value.

9.3.6 TOKEN_PARAM_CONNECTED_PADS key

Available from server version 1.0.0.0 onwards.

The value of the `"TOKEN_PARAM_CONNECTED_PADS"` key is an array that holds the properties of the connected devices. The array may contain the following keys:

- `"TOKEN_PARAM_PAD_INDEX"`
- `"TOKEN_PARAM_PAD_TYPE"`
- `"TOKEN_PARAM_PAD_CONNECTION_TYPE"`
- `"TOKEN_PARAM_PAD_COM_PORT"`
- `"TOKEN_PARAM_PAD_IP_ADDRESS"`
- `"TOKEN_PARAM_PAD_SERIAL_NUMBER"`
- `"TOKEN_PARAM_PAD_FIRMWARE_VERSION"`

The following keys are only available as of server version 1.3.0.0:

- `"TOKEN_PARAM_PAD_CAPABILITIES"`

Example:


```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_SEARCH_FOR_PADS", "TOKEN_PARAM_RETURN_CODE": "16320",
"TOKEN_PARAM_CONNECTED_PADS": [{"TOKEN_PARAM_PAD_INDEX": "0",
"TOKEN_PARAM_PAD_IP_ADDRESS": "", "TOKEN_PARAM_PAD_CONNECTION_TYPE": "2",
"TOKEN_PARAM_PAD_SERIAL_NUMBER": "5", "TOKEN_PARAM_PAD_TYPE": "2",
"TOKEN_PARAM_PAD_FIRMWARE_VERSION": "1.16", "TOKEN_PARAM_PAD_COM_PORT":
"4", "TOKEN_PARAM_PAD_CAPABILITIES": "7874"}, {"TOKEN_PARAM_PAD_INDEX":
"1", "TOKEN_PARAM_PAD_IP_ADDRESS": "", "TOKEN_PARAM_PAD_CONNECTION_TYPE":
"0", "TOKEN_PARAM_PAD_SERIAL_NUMBER": "1000000004",
"TOKEN_PARAM_PAD_TYPE": "1", "TOKEN_PARAM_PAD_FIRMWARE_VERSION": "1.22",
"TOKEN_PARAM_PAD_COM_PORT": "0", "TOKEN_PARAM_PAD_CAPABILITIES":
"16320"}]}
```


9.3.7 TOKEN_PARAM_PAD_INDEX key

Available from server version 1.0.0.0 onwards.

The value of the `"TOKEN_PARAM_PAD_INDEX"` key is an integer that represents the unique ID of a connected device. The ID remains valid until a new request of type `"TOKEN_CMD_SEARCH_FOR_PADS"` is sent.

For an example, see the `"TOKEN_CMD_OPEN_PAD"` value.

9.3.8 TOKEN_PARAM_PAD_TYPE key

Available from server version 1.0.0.0 onwards.

The value of the `"TOKEN_PARAM_PAD_TYPE"` key holds the model number of a connected device. For details, please refer to section ‘Model numbers’.

For an example, see the `"TOKEN_PARAM_CONNECTED_PADS"` key.

9.3.9 TOKEN_PARAM_PAD_CONNECTION_TYPE key

Available from server version 1.0.0.0 onwards.

The value of the `"TOKEN_PARAM_PAD_CONNECTION_TYPE"` key contains the type of connection via which a device is connected. For details, please refer to section ‘Connection types’.

For an example, see the `"TOKEN_PARAM_CONNECTED_PADS"` key.

9.3.10 TOKEN_PARAM_PAD_COM_PORT key

Available from server version 1.0.0.0 onwards.

The value of the `"TOKEN_PARAM_PAD_COM_PORT"` key contains the number of the COM port to which a device is connected.

The `"TOKEN_PARAM_PAD_COM_PORT"` key is not implemented under Linux and has the value 0.

For an example, see the `"TOKEN_PARAM_CONNECTED_PADS"` key.

9.3.11 TOKEN_PARAM_PAD_IP_ADDRESS key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_IP_ADDRESS" key holds the IP address of a connected device.

The "TOKEN_PARAM_PAD_IP_ADDRESS" key is not implemented under Linux and has the value "".

For an example, see the "TOKEN_PARAM_CONNECTED_PADS" key.

9.3.12 TOKEN_PARAM_PAD_SERIAL_NUMBER key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_SERIAL_NUMBER" key holds the serial number of a connected device.

For an example, see the "TOKEN_PARAM_CONNECTED_PADS" key.

9.3.13 TOKEN_PARAM_PAD_FIRMWARE_VERSION key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_FIRMWARE_VERSION" key holds the firmware version (major.minor) of a connected device.

For an example, see the "TOKEN_PARAM_CONNECTED_PADS" key.

9.3.14 TOKEN_PARAM_FIELD_NAME key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_FIELD_NAME" key can contain any character string that is output at the top left of the signature device's display. This should usually be the name of the signature field that is to be signed.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.15 TOKEN_PARAM_CUSTOM_TEXT key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_CUSTOM_TEXT" key can contain any character string that is output at the bottom left of the signature device's display.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.16 TOKEN_PARAM_TEXT_LAYOUT key

Available from server version 1.7.0.0 onwards.

The "TOKEN_PARAM_TEXT_LAYOUT" key determines any texts in a variable layout that are displayed when signing on a signature device. Its value must be an array which can contain the following keys:

- "TOKEN_PARAM_LAYOUT_ID"
- "TOKEN_PARAM_TEXT_BLOCKS"

The "TOKEN_PARAM_TEXT_LAYOUT" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.17 TOKEN_PARAM_LAYOUT_ID key

Available from server version 1.7.0.0 onwards.

The "TOKEN_PARAM_LAYOUT_ID" key determines an integer ID for a text layout. An individual image memory is reserved for every ID in Omega, Gamma, Delta and Alpha devices in which texts can be permanently stored. This allows various layouts with constant texts to be stored in the device. The maximum number of layouts depends on the device and its configuration. If there are no more memories available, the texts will be automatically retransferred with every display.

The "TOKEN_PARAM_LAYOUT_ID" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.18 TOKEN_PARAM_TEXT_BLOCKS key

Available from server version 1.7.0.0 onwards.

The "TOKEN_PARAM_TEXT_BLOCKS" key defines a text block within a layout. Its value must be an array which can contain the following keys:

- "TOKEN_PARAM_TEXT"
- "TOKEN_PARAM_WIDTH"
- "TOKEN_PARAM_HEIGHT"
- "TOKEN_PARAM_FONT_NAME"
- "TOKEN_PARAM_FONT_STYLE"
- "TOKEN_PARAM_MAX_FONT_SIZE"
- "TOKEN_PARAM_FONT_SIZE_ID"
- "TOKEN_PARAM_CONSTANT"

The "TOKEN_PARAM_TEXT_BLOCKS" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.19 TOKEN_PARAM_TEXT key

Available from server version 1.7.0.0 onwards.

The "TOKEN_PARAM_TEXT" key defines the text of text block. It may have any length and contain "\n" for a new paragraph.

The "TOKEN_PARAM_TEXT" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.20 TOKEN_PARAM_WIDTH key

Available from server version 1.7.0.0 onwards.

The "TOKEN_PARAM_WIDTH" key defines the width of a text block in per cent; decimal points are permitted. "100" means that the total available width will be used.

The "TOKEN_PARAM_WIDTH" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.21 TOKEN_PARAM_HEIGHT key

Available from server version 1.7.0.0 onwards.

The "TOKEN_PARAM_HEIGHT" key defines the height of a text block in per cent; decimal points are permitted. "100" means that the total available height will be used.

The "TOKEN_PARAM_HEIGHT" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.22 TOKEN_PARAM_FONT_NAME key

Available from server version 1.7.0.0 onwards.

The "TOKEN_PARAM_FONT_NAME" key defines the font of a text block. This font must be installed on the client; otherwise, 'Arial' is used.

The "TOKEN_PARAM_FONT_NAME" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.23 TOKEN_PARAM_FONT_STYLE key

Available from server version 1.7.0.0 onwards.

The "TOKEN_PARAM_FONT_STYLE" key defines the font style of a text block. It can contain an array of the strings "BOLD", "ITALIC" and "UNDERLINE". If the key is missing or does not contain a valid value, the standard style of the font is used.

The "TOKEN_PARAM_FONT_STYLE" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.24 TOKEN_PARAM_MAX_FONT_SIZE key

Available from server version 1.7.0.0 onwards.

The "TOKEN_PARAM_MAX_FONT_SIZE" key defines the maximum size for text output in per cent; decimal places are permitted. "100" is equivalent to 20 pt for Sigma and Zeta devices,

and to 40 pt for Gamma, Delta, Omega, and Alpha devices; a smaller size (up to a minimum of 12 pt) is used for long texts in order for the text to be fully displayed.

The "TOKEN_PARAM_MAX_FONT_SIZE" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.25 TOKEN_PARAM_FONT_SIZE_ID key

Available from server version 1.7.0.0 onwards.

The "TOKEN_PARAM_FONT_SIZE_ID" key determines an integer ID. All text blocks for which the same ID is used will then use the same font size (the smallest necessary to fit the texts into all text blocks).

The "TOKEN_PARAM_FONT_SIZE_ID" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.26 TOKEN_PARAM_CONSTANT key

Available from server version 1.7.0.0 onwards.

The "TOKEN_PARAM_CONSTANT" key determines whether a text block rarely changes within a text layout and, if needed, is to be permanently stored in the signature device in order to accelerate display.

Permitted values are:

Value	Description
"TRUE"	Text is constant
"FALSE"	Text is not constant (standard)

Note: Text blocks marked as constant are not displayed if the parameter "TOKEN_PARAM_PAD_DIALOG_IMAGE" is also used.

The "TOKEN_PARAM_CONSTANT" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.27 TOKEN_PARAM_PAD_ENCRYPTION key

Available from server version 1.3.0.0 onwards.

The "TOKEN_PARAM_PAD_ENCRYPTION" key sets 'RSA in pad' mode. If it contains "TRUE", the signoPAD API/Web will be switched to 'RSA in pad' mode. 'RSA in pad' mode remains disabled in all other cases. Before the server is switched to 'RSA in pad' mode, the device properties are checked to determine if the device supports encryption in the pad. A check is then performed (based on whether the parameter "TOKEN_PARAM_ENCRYPTION_CERT" was transferred) to determine if the pad has stored a biometric key or attempts to upload the key transferred with "TOKEN_PARAM_ENCRYPTION_CERT" to the pad. An error is generated if it is determined that encryption is not possible in the pad.

The "TOKEN_PARAM_PAD_ENCRYPTION" key is not implemented in Linux.

For examples, see the "TOKEN_CMD_SIGNATURE_START" values.

9.3.28 TOKEN_PARAM_DOCHASH key

Available from server version 1.3.0.0 onwards.

The "TOKEN_PARAM_DOCHASH" key is only evaluated if the "TOKEN_PARAM_PAD_ENCRYPTION" key contains the value "TRUE". The key "TOKEN_PARAM_DOCHASH" contains a 20-byte (SHA1) or 32-byte (SHA256) byte array as a Base64 string that is passed to the pad; the hash is encrypted together with the biometric data and may be signed – see section 'Signing biometric data'. Usually this is the checksum of a document to be signed.

The "TOKEN_PARAM_DOCHASH" key is not implemented in Linux.

For examples, see the "TOKEN_CMD_SIGNATURE_START" values.

9.3.29 TOKEN_PARAM_ENCRYPTION_CERT key

Available from server version 1.3.0.0 onwards.

The "TOKEN_PARAM_ENCRYPTION_CERT" key is only evaluated if the "TOKEN_PARAM_PAD_ENCRYPTION" key contains the value "TRUE". The "TOKEN_PARAM_ENCRYPTION_CERT" key contains a Base64 string that is an X.509 certificate when decoded. The certificate contains the public key used to encrypt the biometric data on the pad. The enclosed certificate the key contains is uploaded to the pad.

The "TOKEN_PARAM_ENCRYPTION_CERT" key is not implemented in Linux.

For examples, see the "TOKEN_CMD_SIGNATURE_START" values.

9.3.30 TOKEN_PARAM_ENCRYPTION_CERT_ONLY_WHEN_EMPTY key

Available from server version 3.2.0.0 onwards.

Specifies whether the public key sent in the "TOKEN_PARAM_ENCRYPTION_CERT" key is only loaded in the pad if the pad does not yet contain one; ignored if the key "TOKEN_PARAM_ENCRYPTION_CERT" is not set.

Permitted values are:

Value	Description
"TRUE"	The public key for encrypting the biometric data is only loaded in the pad if the pad does not yet contain one
"FALSE"	The public key for encrypting the biometric data is always loaded in the pad (default)

The "TOKEN_PARAM_ENCRYPTION_CERT_ONLY_WHEN_EMPTY" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.31 TOKEN_PARAM_PAD_DIALOG_IMAGE key

Available from server version 1.6.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_DIALOG_IMAGE" key can hold a Base64 encoded image that is output below the toolbar on the signature device's display. The image should be of the size transmitted in the "TOKEN_PARAM_PAD_DIALOG_WIDTH" and "TOKEN_PARAM_PAD_DIALOG_HEIGHT" keys.

If one of the "TOKEN_PARAM_FIELD_NAME", "TOKEN_PARAM_CUSTOM_TEXT" or "TOKEN_PARAM_TEXT_LAYOUT" keys is also filled, the text is output on the transferred image and framed by a white rectangle.

The "TOKEN_PARAM_PAD_DIALOG_IMAGE" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_START" value.

9.3.32 TOKEN_PARAM_SIGNATURE_SIGN_DATA key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_SIGNATURE_SIGN_DATA" key holds the SignData of a captured signature. SignData is an encrypted, compressed, biometric data format that can be stored in a database or as a tag in a TIFF or PDF document.

For an example, see the "TOKEN_CMD_SIGNATURE_SIGN_DATA" value.

9.3.33 TOKEN_PARAM_SIGNATURE_RSA_SCHEME key

Available from server version 3.2.0.0 onwards.

The value of the "TOKEN_PARAM_SIGNATURE_RSA_SCHEME" key determines the signature scheme of the biometric data signature for a captured signature. For details, please refer to section 'Signing biometric data'.

For an example, see the "TOKEN_CMD_SIGNATURE_SIGN_DATA" value.

9.3.34 TOKEN_PARAM_SIGNATURE_RSA_SIGNATURE key

Available from server version 3.2.0.0 onwards.

The value of the " TOKEN_PARAM_SIGNATURE_RSA_SIGNATURE" key contains the biometric data signature for a captured signature. The signature is generated with a private key that is saved in the pad. The data that are signed are either a combination of Hash 1 and Hash 2, or Hash 2.

For an example, see the "TOKEN_CMD_SIGNATURE_SIGN_DATA" value.

9.3.35 TOKEN_PARAM_CERT_ID key

Available from server version 1.3.0.0 onwards.

The "TOKEN_PARAM_CERT_ID" key is only returned if the "TOKEN_PARAM_PAD_ENCRYPTION" key contains the value "TRUE". The returned value is the ID of the public key that is stored in the signature device and used for the encryption of biometric data. The ID is a character string consisting of the 'Issuer' and 'Serial Number' of the certificate in hexadecimal representation that has been transferred when saving the key, e.g., 'Example certificate (ff 0a)'.

The "TOKEN_PARAM_CERT_ID" key is not implemented in Linux.

For an example, see the "TOKEN_CMD_SIGNATURE_SIGN_DATA" value.

9.3.36 TOKEN_PARAM_FILE_TYPE key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_FILE_TYPE" key specifies a file type. For details, please refer to section 'Image formats'.

For an example, see the "TOKEN_CMD_SIGNATURE_IMAGE" value.

9.3.37 TOKEN_PARAM_PEN_WIDTH key

Available from server version 1.1.0.0 onwards.

The value of the "TOKEN_PARAM_PEN_WIDTH" key specifies a line width when generating signature images. For details, please refer to section 'Pen widths'.

For an example, see the "TOKEN_CMD_SIGNATURE_IMAGE" value.

9.3.38 TOKEN_PARAM_FILE key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_FILE" key holds a file as a Base64-encoded string.

Example:


```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_SIGNATURE_IMAGE", "TOKEN_PARAM_RETURN_CODE": "0",
"TOKEN_PARAM_FILE":
"CgAAAAEAAABvAAgCVQVVBVUVABAEAAEAAQAABA2IYa1x2GVCVU4WzHpbdUCrrInm
GTLTbzfOOd/WFQo47vAi9Af4AFc="}
```


9.3.39 TOKEN_PARAM_SIGNING_CERT key

Available from server version 1.7.0.0 onwards.

The value of the "TOKEN_PARAM_SIGNING_CERT" key contains the public X.509 certificate saved in the signature device, the corresponding private key of which is used for signing data, in Base64-coded form.

Example:


```
{"TOKEN_TYPE": "TOKEN_TYPE_RESPONSE", "TOKEN_CMD_ORIGIN":
"TOKEN_CMD_SIGNING_CERT", "TOKEN_PARAM_RETURN_CODE": "0",
"TOKEN_PARAM_SIGNING_CERT":
"MIIBizCB9aADAgECAgEBMA0GCSqGSIb3DQEBBQUAMAwxCjAIBgNVBAMTATEwHhcN
MTkwNTEwMTIyMDAwWhcNMjAwNTEwMTIyMDAwWjAMMQowCAYDVQQDEwExMIGfMA0G
CSqGSIb3DQEBAQUAA4GNADCBiQKBgQCcLHRsAVu9Ej6GHlLppULv5RPWMzZwadl2
2793l6bLn/Bhjmub/uig8C1NnPL1ByEwplCoSwW0UgOmKxODEcfn7PpzzaLvzkKp
8ViDVwdhGi3TQECSx9XBdRLzybi29GG+++k3xcrwXaOkkOJZgIS/23qCc4BGBrJG
Fie9JctlPwIDAQABMA0GCSqGSIb3DQEBBQUAA4GBAEa8LLS1WD2R2R/XSvGkmpgT
94sqzKMlzxPmRnoXSp4uPrk0JvseSu+kjidIBWFDmQX72EArx5c9t0WmuONlZIyU
htcgeWpd+qv9Vq9R1AW9JJ2Z9EgkTrff51UI2oZtis7BuzZ0jOUCKCDgB1nczmwG
lkeeep4LBqZV0oBIZQzo"}
```


9.3.40 TOKEN_PARAM_EXCEPTION_CAUSE key

Available from server version 3.1.0.0 onwards.

The value of the `"TOKEN_PARAM_EXCEPTION_CAUSE"` key contains the exception text that occurs when parsing an incorrect command.

For an example, see the `"TOKEN_CMD_INCORRECT_COMMAND"` value.

9.3.41 TOKEN_PARAM_POINT key

Available from server version 1.0.0.0 onwards.

The value of the `"TOKEN_PARAM_POINT"` key holds an object with information about a captured signature point. The resolution of the data is not suitable for use as biometric data. Please use SignData instead. You can retrieve this using the `"TOKEN_CMD_SIGNATURE_SIGN_DATA"` value.

The object may contain the following keys:

Key	Description
`"x"`	X-coordinate between `"0"` and the display width or horizontal signature resolution of the pad model used (see also `"TOKEN_PARAM_PAD_DISPLAY_WIDTH"` and `"TOKEN_PARAM_PAD_X_RESOLUTION"` keys).
`"y"`	Y-coordinate between `"0"` and the display height or vertical signature resolution of the pad model used (see also `"TOKEN_PARAM_PAD_DISPLAY_HEIGHT"` and `"TOKEN_PARAM_PAD_Y_RESOLUTION"` keys).
`"p"`	`"0"` defines the point of setting down the pen (first point of a signature line).
	`"1"` defines a pen that is set down (all further points of a signature line).

For an example, see the `"TOKEN_CMD_SIGNATURE_POINT"` value.

9.3.42 TOKEN_PARAM_OS key

Available from server version 1.6.0.0 onwards.

The value of the `"TOKEN_PARAM_OS"` key holds either `"Windows"` or `"Linux"`.

For an example, see the "TOKEN_CMD_GET_SERVER_VERSION" value.

9.3.43 TOKEN_PARAM_VERSION key

Available from server version 1.0.2.4 onwards.

The value of the "TOKEN_PARAM_VERSION" key must be in the format "major.minor.revision.build".

Permitted values are:

Part	Permitted values
"major"	0–65535
"minor"	0–65535
"revision"	0–65535
"build"	0–65535

For the "TOKEN_CMD_SET_INTERFACE_VERSION" command, the parameter sets the version for the supported server interface. For the "TOKEN_CMD_GET_INTERFACE_VERSION" command, the parameter returns the set version of the server interface. For the "TOKEN_CMD_GET_SERVER_VERSION" command, the parameter returns the server version.

For examples, see the "TOKEN_CMD_SET_INTERFACE_VERSION" and "TOKEN_CMD_GET_SERVER_VERSION" values.

9.3.44 TOKEN_PARAM_PAD_DISPLAY_WIDTH key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_DISPLAY_WIDTH" key is sent with the response to the "TOKEN_CMD_OPEN_PAD" and "TOKEN_CMD_API_DISPLAY_GET_WIDTH" commands. The value of the "TOKEN_PARAM_PAD_DISPLAY_WIDTH" key is the width of the screen of a signotec LCD signature pad. The X-coordinate in the "TOKEN_CMD_SIGNATURE_POINT" event relates to the value of the "TOKEN_PARAM_PAD_DISPLAY_WIDTH" key up to server version 1.0.2.3.

For examples, see the "TOKEN_CMD_OPEN_PAD" and "TOKEN_CMD_SIGNATURE_POINT" values.

9.3.45 TOKEN_PARAM_PAD_DISPLAY_HEIGHT key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_DISPLAY_HEIGHT" key is sent with the response to the "TOKEN_CMD_OPEN_PAD" and "TOKEN_CMD_API_DISPLAY_GET_HEIGHT" commands. The value of the "TOKEN_PARAM_PAD_DISPLAY_HEIGHT" key is the width of the screen of a signotec LCD signature pad. The Y-coordinate in the "TOKEN_CMD_SIGNATURE_POINT" event relates to the value of the "TOKEN_PARAM_PAD_DISPLAY_HEIGHT" key up to server version 1.0.2.3.

For examples, see the "TOKEN_CMD_OPEN_PAD" and "TOKEN_CMD_SIGNATURE_POINT" values.

9.3.46 TOKEN_PARAM_PAD_SAMPLING_RATE key

Available from server version 1.0.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_SAMPLING_RATE" key is sent with the response to the "TOKEN_CMD_OPEN_PAD" command and holds the sampling rate of the pad in Hz.

For an example, see the "TOKEN_CMD_OPEN_PAD" value.

9.3.47 TOKEN_PARAM_PAD_X_RESOLUTION key

Available from server version 1.0.2.4 onwards.

The value of the "TOKEN_PARAM_PAD_X_RESOLUTION" key is sent with the response to the "TOKEN_CMD_OPEN_PAD" and "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION" commands. The value of the "TOKEN_PARAM_PAD_X_RESOLUTION" key is the X resolution of the virtual screen for the real-time signature display. The X-coordinate in the "TOKEN_CMD_SIGNATURE_POINT" event relates to the value of the "TOKEN_PARAM_PAD_X_RESOLUTION" key from server version 1.0.2.4 onwards.

For examples, see the "TOKEN_CMD_OPEN_PAD", "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION" and "TOKEN_CMD_SIGNATURE_POINT" values.

9.3.48 TOKEN_PARAM_PAD_Y_RESOLUTION key

Available from server version 1.0.2.4 onwards.

The value of the "TOKEN_PARAM_PAD_Y_RESOLUTION" key is sent with the response to the "TOKEN_CMD_OPEN_PAD" and "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION" commands. The value of the "TOKEN_PARAM_PAD_Y_RESOLUTION" key is the Y resolution of the virtual screen for the real-time signature display. The Y-coordinate in the "TOKEN_CMD_SIGNATURE_POINT" event relates to the value of the "TOKEN_PARAM_PAD_Y_RESOLUTION" key from server version 1.0.2.4.

For examples, see the "TOKEN_CMD_OPEN_PAD", "TOKEN_CMD_API_SIGNATURE_GET_RESOLUTION" and "TOKEN_CMD_SIGNATURE_POINT" values.

9.3.49 TOKEN_PARAM_PAD_DIALOG_WIDTH key

Available from server version 1.6.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_DIALOG_WIDTH" key is sent with the response to the "TOKEN_CMD_OPEN_PAD" command. The value of the "TOKEN_PARAM_PAD_DIALOG_WIDTH" key is the width of the image that can be passed in the "TOKEN_PARAM_PAD_DIALOG_IMAGE" parameter when using the "TOKEN_CMD_SIGNATURE_START" command.

For an example, see the "TOKEN_CMD_OPEN_PAD" value.

9.3.50 TOKEN_PARAM_PAD_DIALOG_HEIGHT key

Available from server version 1.6.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_DIALOG_HEIGHT" key is sent with the response to the "TOKEN_CMD_OPEN_PAD" command. The value of the "TOKEN_PARAM_PAD_DIALOG_HEIGHT" key represents the height of the image that can be passed in the "TOKEN_PARAM_PAD_DIALOG_IMAGE" parameter when using the "TOKEN_CMD_SIGNATURE_START" command.

For an example, see the "TOKEN_CMD_OPEN_PAD" value.

9.3.51 TOKEN_PARAM_PAD_DISPLAY_RESOLUTION key

Available from server version 1.6.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_DISPLAY_RESOLUTION" key is sent with the response to the "TOKEN_CMD_OPEN_PAD" command. The value of the "TOKEN_PARAM_PAD_DISPLAY_RESOLUTION" key is the display resolution of a signotec LCD signature pad in pixels per inch (ppi).

For an example, see the "TOKEN_CMD_OPEN_PAD" value.

9.3.52 TOKEN_PARAM_PAD_CAPABILITIES key

Available from server version 1.3.0.0 onwards.

The value of the "TOKEN_PARAM_PAD_CAPABILITIES" key is sent with the response to the "TOKEN_CMD_SEARCH_FOR_PADS" command. The value of the "TOKEN_PARAM_PAD_CAPABILITIES" key is a bit field (integer) that describes the properties of a device. For details, please refer to section 'Device properties'.

For examples, see the "TOKEN_PARAM_CONNECTED_PADS" key.

9.3.53 TOKEN_PARAM_FIELD_LIST key

Available from server version 1.4.0.0 onwards.

The "TOKEN_PARAM_FIELD_LIST" key determines the selection fields and their texts for the selection dialog configurable with the "TOKEN_CMD_SELECTION_DIALOG" command. Its value must be an array which can contain the following keys:

- "TOKEN_PARAM_FIELD_ID"
- "TOKEN_PARAM_FIELD_TEXT"
- "TOKEN_PARAM_FIELD_CHECKED"
- "TOKEN_PARAM_FIELD_REQUIRED"

For an example, see the "TOKEN_CMD_SELECTION_DIALOG" value.

9.3.54 TOKEN_PARAM_FIELD_ID key

Available from server version 1.4.0.0 onwards.

The "TOKEN_PARAM_FIELD_ID" key determines the ID of a selection field or contains the pre-defined ID. The ID can be any value, but may not have already been used for another field.

For examples, see the "TOKEN_CMD_SELECTION_DIALOG" and "TOKEN_CMD_SELECTION_CHANGE" values.

9.3.55 TOKEN_PARAM_FIELD_TEXT key

Available from server version 1.4.0.0 onwards.

The "TOKEN_PARAM_FIELD_TEXT" key determines the text of a selection field. It may have any length and contain "\n" for a new paragraph.

For an example, see the "TOKEN_CMD_SELECTION_DIALOG" value.

9.3.56 TOKEN_PARAM_FIELD_CHECKED key

Available from server version 1.4.0.0 onwards.

The "TOKEN_PARAM_FIELD_CHECKED" key determines whether a selection field is selected.

Permitted values:

Value	Description
"TRUE"	Field is selected.
"FALSE"	Field is not selected.

For examples, see the "TOKEN_CMD_SELECTION_DIALOG" and "TOKEN_CMD_SELECTION_CHANGE" values.

9.3.57 TOKEN_PARAM_FIELD_REQUIRED key

Available from server version 1.4.0.0 onwards.

The "TOKEN_PARAM_FIELD_REQUIRED" key determines whether a selection field must be selected to be able to confirm the selection dialog. If the 'Confirm' button is activated without all mandatory fields having been selected, an event with the command "TOKEN_CMD_ERROR" and "TOKEN_PARAM_RETURN_CODE" "-400" is triggered.

Permitted values:

Value	Description
"TRUE"	Field must be selected (mandatory field).
"FALSE"	Field does not have to be selected (optional field).

For an example, see the "TOKEN_CMD_SELECTION_DIALOG" value.

9.3.58 TOKEN_PARAM_ROTATION key

Available from server version 1.3.0.0 onwards.

The "TOKEN_PARAM_ROTATION" key defines the rotation with which the image data is transferred to the signature device and the sensor areas are defined. A change to the value only affects images and sensor areas which are transferred after the change is made and not images which are copied from one pad memory to another. The default setting is 0.

For examples, see the "TOKEN_CMD_API_DISPLAY_SET_ROTATION" and "TOKEN_CMD_API_DISPLAY_GET_ROTATION" values.

9.3.59 Other keys

All other keys can be used as request parameters in API mode. For a description, please see the corresponding method in the signoPAD API documentation.

10 Common definitions for both integration variants

Definitions that apply to both integration variants are given below.

10.1 Error codes

The following error codes are possible:

Error	Description
-3	One of the parameters that were passed contains an invalid value.
-4	The signing pad is already being used by another application.
-5	No connection has been opened to this signature pad.
-6	A connection has already been opened.
-7	No further connections can be opened.
-8	No device with this ID is connected.
-12	The function could not be executed because the signature capture process is running.
-13	No further hotspots can be added
-14	The coordinates are overlapping with the signature window or one of the hotspots already set.
-15	The function could not be executed because no signature capture area has been set.
-17	The function could not be executed because no signature capture process was started.
-18	An error occurred while attempting to reserve memory.
-19	An error occurred while initialising a system resource.
-20	An error occurred while communicating with the signing pad.
-21	The rectangle that was passed is invalid.
-22	No compatible devices connected or the connection to a device has been cut.
-25	The connected device does not support this function or one of the parameters.
-26	Error while reading or writing a file.
-46	The certificate or one of the certificates in the certificate chain has expired or is not yet valid.
-47	Trust for the certificate or one of the certificates in the certificate chain has been revoked.
-48	The certificate or one of the certificates in the certificate chain does not have a valid signature.
-84	The function could not be executed because the secure sign mode is active.
-93	The function could not be executed because an overlay rectangle is set.
-94	The function could not be executed because the display content is scrolled.
-95	The function could not be executed because it would have activated the scroll mode that is not possible if a hotspot outside the overlay rectangle is defined.
-97	An error occurred during initialisation. Please restart the software.
-400	Not all mandatory fields were selected.

10.2 List for pad search

When searching for signature devices, it is possible to define which connection types should be searched for. The individual connection types are passed in a list separated by semi-colons with at least one of the following members:

Values	Description
"HID"	A search will be made for HID and USB devices.
"IP=<Address>:<Port>"	An Alpha or Delta with an Ethernet connection or an HID device that is connected to a signotec AK USB device server will be searched for at the specified IP address; <Address> is the IP address of the device server and has the format x.x.x.x or the host name, <Port> is the port via which the signature device should be activated, i.e. generally 1002. For example, a valid value would therefore be "IP=192.168.100.100:1002" or "IP=host1:1002"; for details about the signotec AK USB device server, please refer to your contact at signotec.
"all"	A search will be made for serial devices on all COM ports; this search may take an extremely long time depending on the hardware configuration.
Integer	A search will be made for a serial device on the COM port with the specified number (>=0); this search may take an extremely long time if no signotec device is connected to the port
"LowSpeed"	A search will be made on the COM ports only for devices that communicate at a baud rate of 115200 (Sigma, Zeta, Omega, Gamma plus Delta and Alpha in 'Low Speed' mode); this accelerates the search if COM ports to which no signotec LCD signature pads are connected are also searched. However, this does not influence the search for HID, WinUSB or IP devices.
"HighSpeed"	A search will be made on the COM ports only for devices that communicate at a baud rate of 2 MBaud (Delta and Alpha in 'High Speed' mode), which accelerates the search; this does not influence the search for HID, WinUSB or IP devices.

In Linux, only HID pads are found.

10.3 Model numbers

The following model numbers are currently supported:

Values	Description
1	'Sigma USB' model type
2	'Sigma serial' model type
5	'Zeta USB' model type
6	'Zeta serial' model type
11	'Omega USB' model type
12	'Omega serial' model type
15	'Gamma USB' model type
16	'Gamma serial' model type
21	'Delta USB' model type
22	'Delta serial' model type
23	'Delta IP' model type
31	'Alpha USB' model type
32	'Alpha serial' model type
33	'Alpha IP' model type
other	Reserved for further model types

10.4 Connection types

The following connection types are currently supported:

Values	Description
0	HID
1	USB
2	Serial
3	Ethernet

10.5 Device properties

Support for a property is indicated by a set bit in the bit field of the device properties. The following properties can be currently supported:

Bit	Description
0x00000001	Device has a colour display
0x00000002	Device has a colour display with backlight
0x00000004	Device supports vertical scrolling
0x00000008	Device supports horizontal scrolling
0x00000010	Device supports scrolling with the pen
0x00000020	The service menu can be called up using the DeviceStartService() method.
0x00000040	Device supports the RSA functions
0x00000080	Device supports content signing
0x00000100	Device supports content signing where only hash 2 is signed
0x00000200	Device can generate a signature key pair
0x00000400	Device can save an externally supplied signature key pair
0x00000800	Device can save an externally supplied biometric key
0x00001000	Device can sign a hash calculated externally (otherwise only a hash calculated through content signing)
0x00002000	RSA keys can be protected through a device password.
0x00004000	The 'Secure mode' can be protected through a device password.
0x00008000	Device supports RSA keys with a length of up to 4096 bits (otherwise up to 2048 bits)
0x00010000	Device has an NFC reader
0x00020000	Device supports keypad encryption with a length of up to 8 characters
0x00040000	Device supports keypad encryption with a length of up to 32 characters
0x00080000	Device has a screen (please note: Support for this property is always displayed even with a Sigma Lite up to firmware 2.3)
0x00100000	The signature key can be protected by a key password

10.6 Image formats

The following image formats are currently supported:

Values	Description
0	Use TIFF with CCITT4 compression (b/w image) or LZW compression (colour image) as the file format (recommended)
1	Use PNG file format
2	Use BMP file format
3	Use JPEG with a quality setting of 75 as the file format
4	Use GIF as the file format

10.7 Signing biometric data

If the biometric data in the pad are encrypted, it is possible to have the data signed digitally by a private key saved on the device in order to guarantee the data's integrity. If the biometric data are embedded in a document, they can be linked to the document in the signature to ensure that they remain together.

First, the hash value of the document to be signed must be calculated (hereafter referred to as hash 1). The SHA-1 and SHA-256 procedures are permissible for this purpose. The hash value of the document Hash 1 is sent in parameter `docHash` of the method `startSignature()` or in the key `"TOKEN_PARAM_DOCHASH"` of the request `"TOKEN_CMD_SIGNATURE_START"`.

When `getSignatureData()` or `"TOKEN_CMD_SIGNATURE_SIGN_DATA"` are called up, the signing of the data can be requested. Depending on whether a Hash 1 was sent at the start of the signature capture process, this is linked with the hash of the biometric data and signed. It is recommended to use the `PSS` signature scheme; alternatively, `NO_HASH_OID` can be used. If no Hash 1 was sent, only the biometric data is signed.

The following values for the signature scheme are supported under Windows for server versions 3.2.0.0 or later:

Values	Description
`"NONE"`	The biometric data are not signed
`"NO_HASH_OID"`	Only a padding is added according to RSASSA-PKCS1-V1_5, no hash OID
`"PKCS1_V1_5"`	The OID of the hash algorithm used and a padding in accordance with RSASSA-PKCS1-V1_5 are added; this scheme can only be used if Hash 1 has not been passed
`"PSS"`	The RSASSA-PSS signature scheme is used; the salt has the same length as Hash 1 and hash 2 respectively, and the algorithm using for encoding is the same as the one that has been used to generate Hash 1 and Hash 2 respectively; MGF 1 is used as mask generation function.

10.8 Pen widths

The following values for pen width are currently supported when generating a signature image:

Values	Description
`< 0`	Fixed pen width in pixels (absolute value); the pressure values are visualised by drawing in variable brightness
`0`	A variable pen width is used that is dependent on the resolution and the pressure values
`> 0`	Fixed pen width in pixels